

**LABORATORY ACTIVITY NO. 2.3.9**

Using Programming Tools

Name: Garcia, Marina Ellise V.**Section:** BSIT-102I

The lab begins by instructing on how to download and install the Python development environment for Windows. It then moves to "Exploring a Simple Program", where users will use Python's IDLE shell and editor to understand basic code constructs. This includes working with *print()* and *input()* functions, variables, loops (specifically the for loop with *range()*), string concatenation, and the importance of indentation in Python. The next section focuses on "Using the Debugger" to identify logic errors by setting breakpoints and stepping through program execution, observing variable values using "Step," "Out," and "Over" functions. Finally, the lab introduces "Using a Function," demonstrating how to incorporate user-defined functions to improve code modularity and grammar, and explores data structures like dictionaries.

I. OBJECTIVES

Upon completion of this laboratory exercise, students are expected to and meet the following intended learning outcomes:

Activity Objectives	Intended Learning Outcomes		
	ILO1	ILO2	ILO3
1) successfully install the Python development environment (IDLE) on a Windows 11 system and write, execute, and understand basic Python programs utilizing variables, input/output functions, loops, and correct indentation.	•	•	
2) effectively use the IDLE debugger in Windows 11 to identify and troubleshoot logic errors in Python code by setting breakpoints, stepping through program execution, and observing variable values.	•	•	
3) implement and utilize custom functions within Python programs on Windows 11 to enhance code modularity and readability, and understand the application of basic data structures such as dictionaries.	•	•	

Where:

ILO1 – Exhibit thorough demonstration of the concept

ILO2 – Being able to perform the activity independently

ILO3 – Design and evaluate emerging IT solutions



II. DISCUSSION

- **Programming Logic**

- Focuses on software development, programming languages, and the importance of clear program sequence, input, processing, and output.
- Illustrates program flow using flowcharts and explains the concept of pseudocode, emphasizing breaking down steps into simple instructions and using subroutines.

- **Features of Programming Code**

- Highlights common constructs across programming languages, including subroutines (user-defined), return statements, conditional statements (if, if-else), variables, functions (pre-defined), user interfaces (console, GUI, web), and comments.

- **Programming Languages**

- Discusses factors in choosing a language, such as program type, programmer experience, and target devices.
- Lists popular languages like Java, Python, JavaScript, C++, C#, PHP, Perl, and Ruby.
- Differentiates between **compiled languages** (transformed to executable binary, fast but platform-specific, e.g., C++, C#) and **interpreted languages** (run within an interpreter, slower but platform-independent, e.g., Java, JavaScript, Python).
- Covers **query languages** (for databases, e.g., SQL), **assembly language** (human-readable machine code), and **markup languages** (for document structure, e.g., HTML, XML).

- **Identifiers**

- Explains variables (values that can change) and constants (values that do not change) in programming.
- Introduces identifiers for multiple elements like arrays (fixed-size, same data type) and vectors (dynamic arrays).

- **Branches and Loops**



- Describes control flow mechanisms: branches (execute code based on condition, e.g., If...Then) and loops (repeat code based on condition, e.g., For...Next, Do While).
- **Operators**
 - Details comparison operators (e.g., ==, !=, <, >) and logical operators (AND, OR, XOR).
- **Procedures, Functions, and Comments**
 - Explains that procedures and functions are used for blocks of code, with functions specifically returning a value.
 - Highlights the use of comments for documenting code, noting they are not executed by the compiler/interpreter.
- **Object-Oriented Programming (OOP)**
 - Introduces the concept of designing code to represent "things" (objects) that interact through defined methods.
 - Defines attributes (values/data types of an object), methods (what can be done to an object), and properties (alternative public access to fields).
- **Scripting Languages**
 - Describes scripting languages as interpreted, often used for OS automation, smaller than programs, and designed for specific tasks (e.g., Batch files, Windows PowerShell, VBScript, Bash).
- **Application Platforms and Delivery Methods**
 - Categorizes applications by platform (single-platform, cross-platform, web applications) and delivery methods (local installation, network installation, cloud-hosted).

III. MATERIALS

A Windows 11 installed computer system.

IV. PROCEDURE

For this activity, you will execute the provided step-by-step instructions. As you complete each step, capture a screenshot of the resulting output or completed action that directly demonstrates successful execution of that instruction. Paste each screenshot into the designated area within your lab document.



Important Considerations:

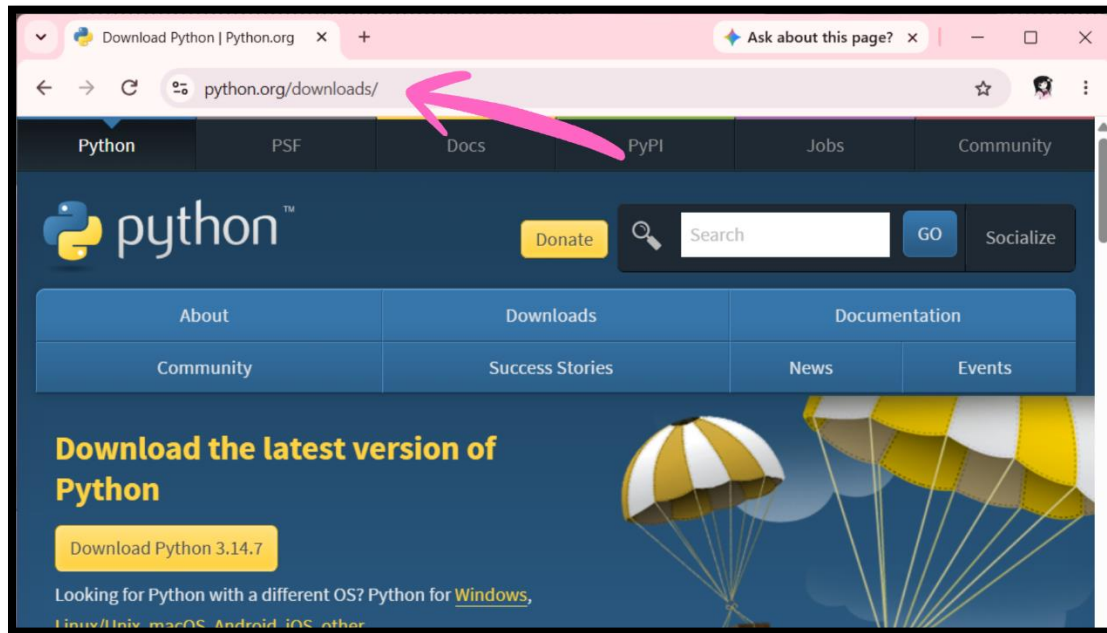
- **Third-Party Software Warning:** This lab involves downloading and installing Python from a third-party website. Please be advised to follow the
- **Python Scripting Language**
 - **Version:** As of July 3, 2025, the latest stable version of Python is **3.13.5**. You should download the latest stable version for Windows from the official Python downloads page.
 - **Installation Options:** During Python installation, ensure you select "Install launcher for all users (recommended)" and consider checking the "Add Python X.Y to PATH" option (where X.Y is your Python version), as this makes it easier to run Python from the command line later.
 - **User Account Control (UAC):** Be prepared to click "Yes" on any User Account Control prompts that appear during the installation process.
 - **Accessing IDLE (Windows 11):** To open IDLE, click the **Start button** (on the taskbar), type idle in the search bar, and then click **IDLE (Python X.X)** from the search results (the specific Python version will be listed).
 - **Python's range() Function:** When exploring loops, remember that Python's range() function is *upper bound exclusive*. For example, range(1, 3, 1) will iterate for 1 and 2, but not 3. To include the end value, you need to add one to the stop value (e.g., range(1, 3+1, 1)).
 - **Indentation in Python:** Pay close attention to indentation. In Python, indentation is crucial for structuring code and is critical for compilation and execution, unlike many other languages where it's primarily for clarity.
- **Debugger Usage:** Familiarize yourself with the debugger controls:
 - **Go:** Continues execution normally until a breakpoint is hit.
 - **Step (Step Into):** Advances through statements and functions step-by-step.
 - **Over (Step Over):** Executes the next statement without pausing for each part of any functions it contains.
 - **Out (Step Out):** Completes any remaining parts within a function and pauses at the next statement in the main program flow.



Steps:

Part 1: Installing Programming Tools

1. Open a web browser and navigate to `www.python.org/downloads`.

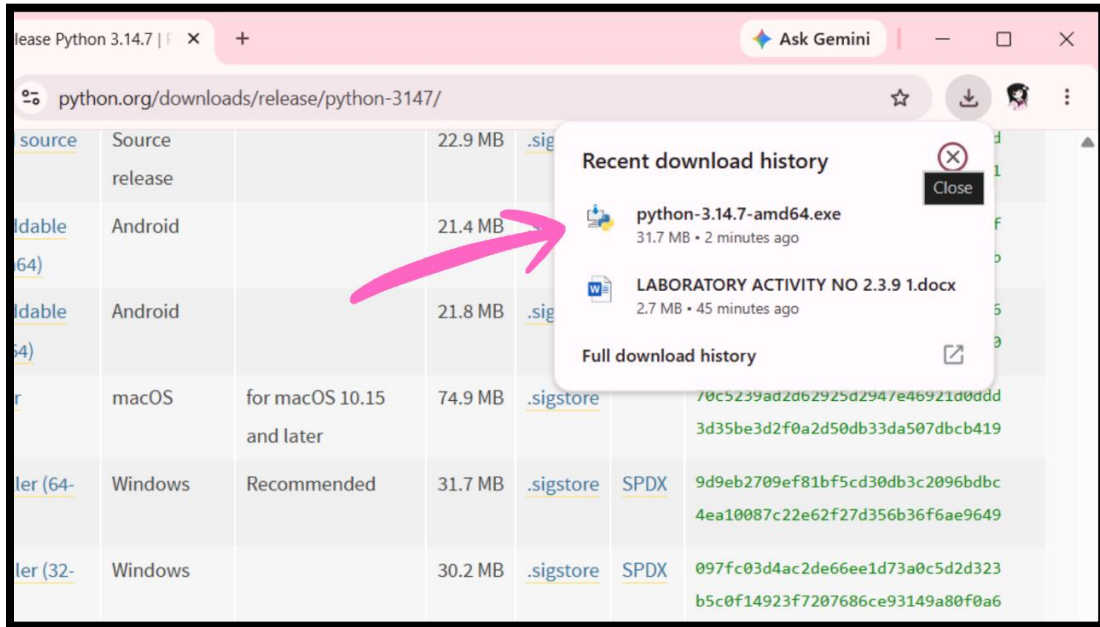


2. Click the **Download Python** button to download the latest version of Python for Windows. Note that the exact version of Python available may vary over time.

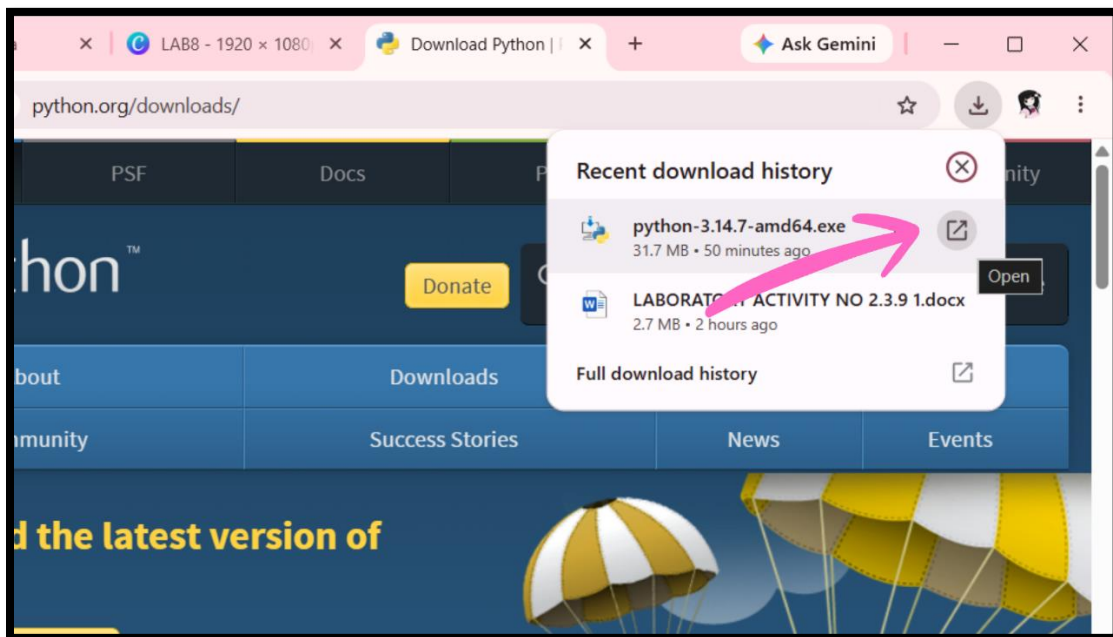




- When prompted, in the download bar at the bottom of the browser window, click the **Save** button.

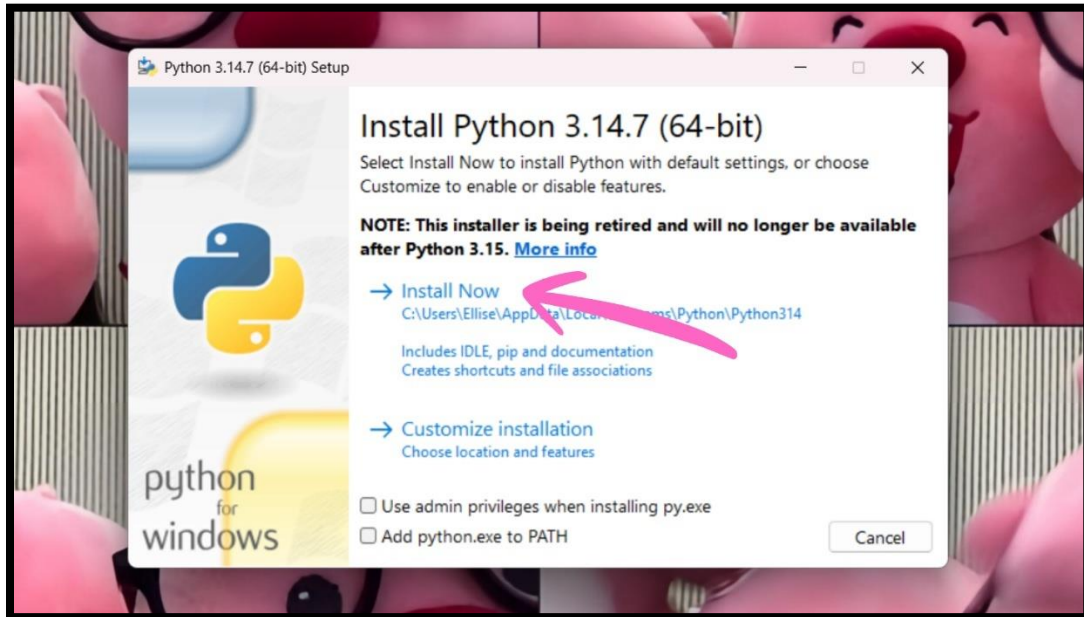


- When the file has downloaded and the security scan is complete, in the download bar, click the **Run** button.

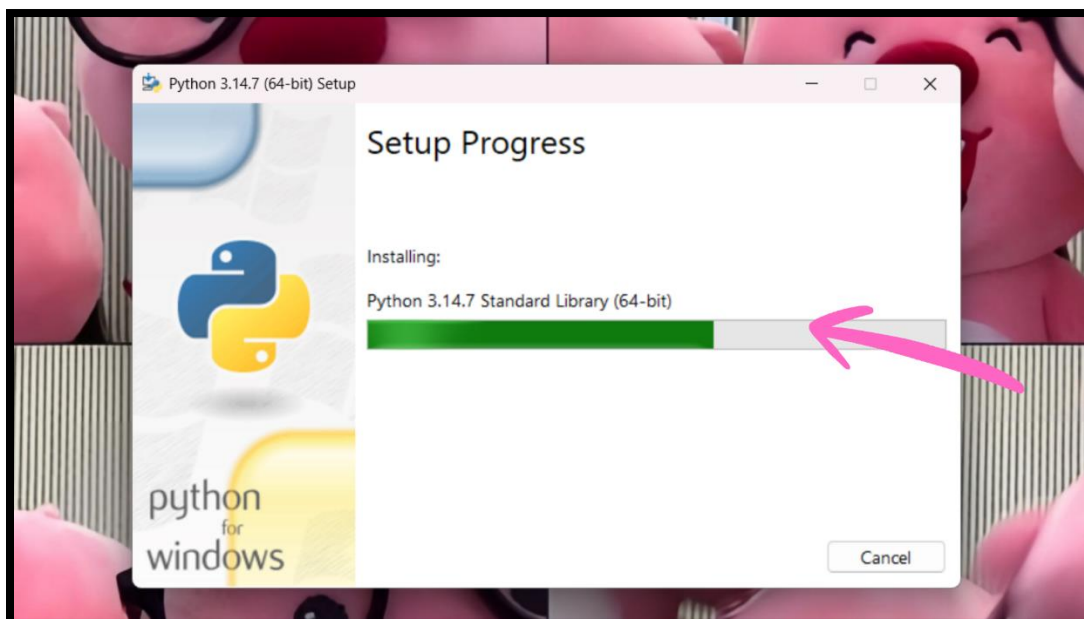




5. In the Python installer window, ensure "Add Python X.Y to PATH" is checked (where X.Y is your Python version, e.g., 3.13) and then click **Install Now**.

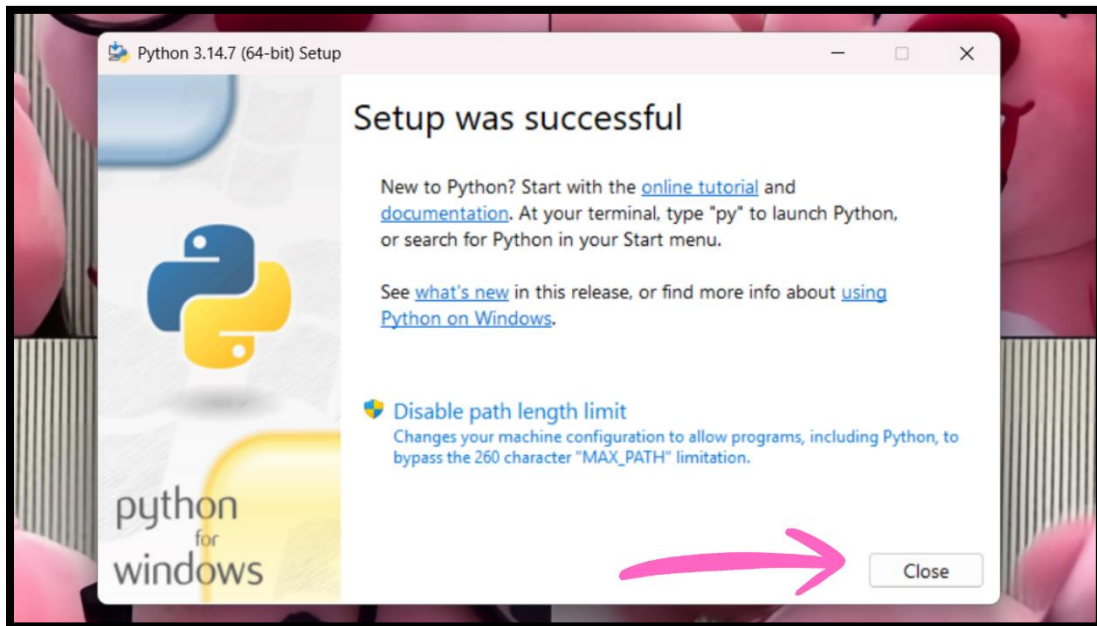


6. In the User Account Control window, click **Yes**.

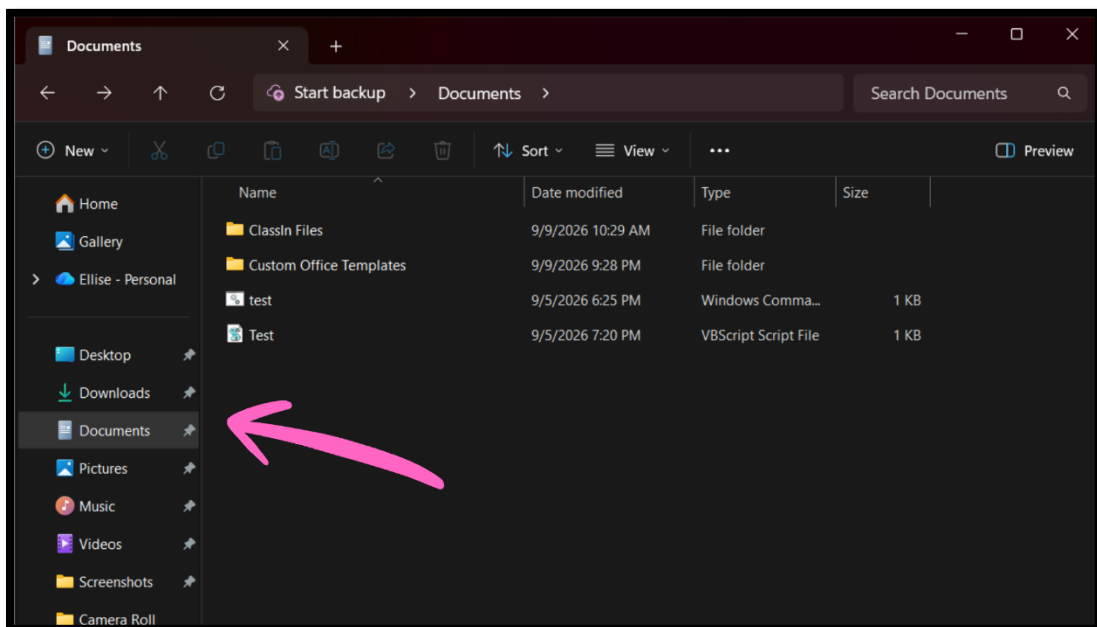




7. Click **Close** when the installation is complete.

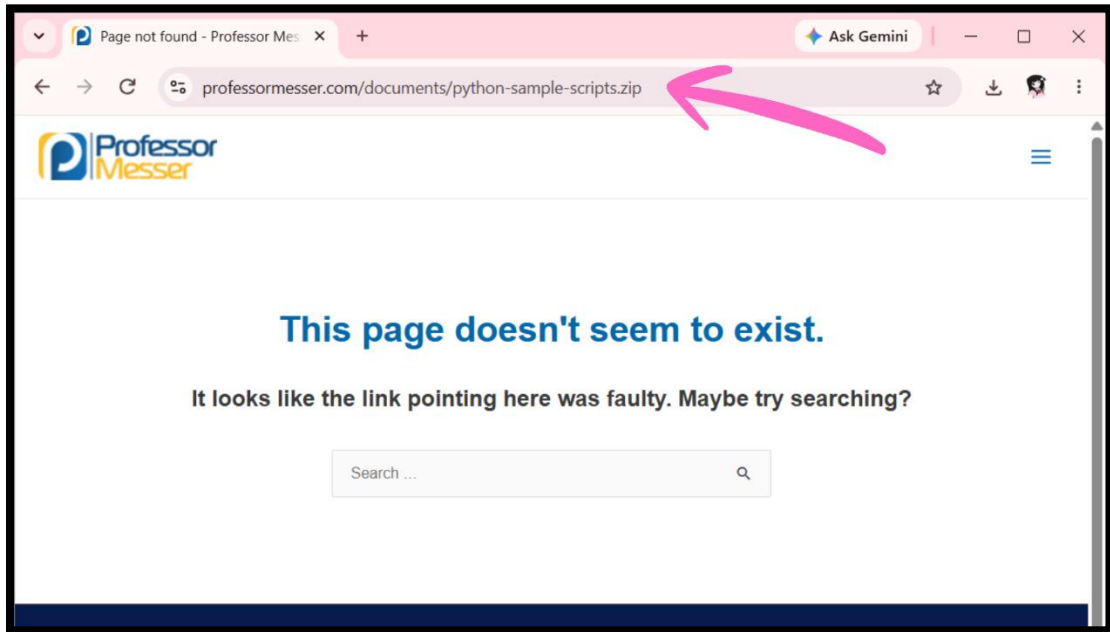


8. Open **File Explorer** (by clicking the folder icon on the taskbar or pressing Windows key + E) and navigate to your **Documents** folder.

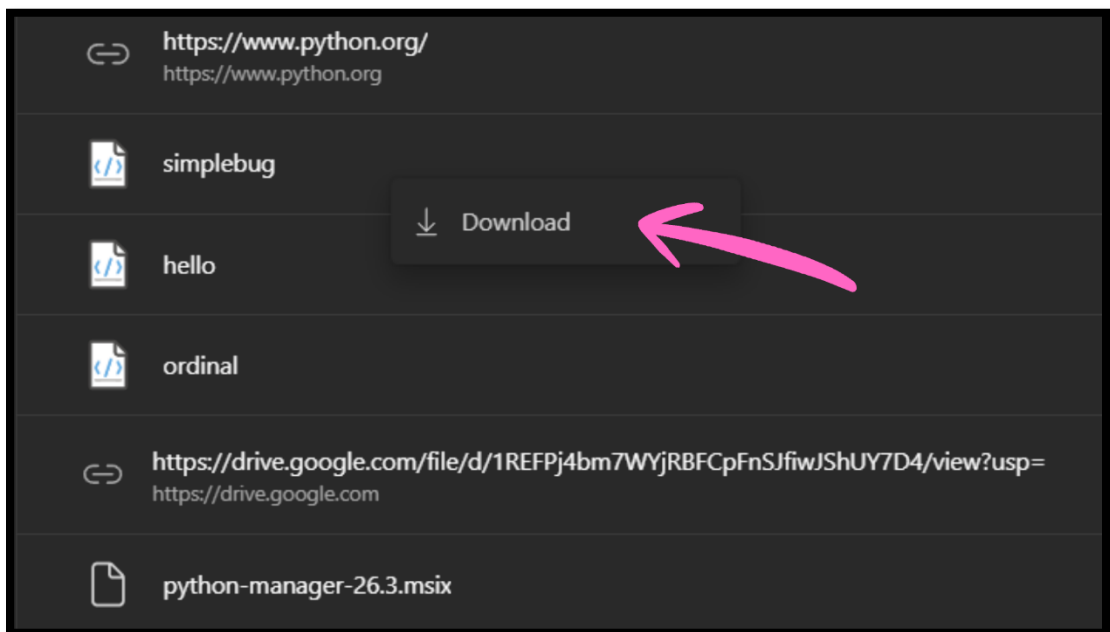




9. In the web browser window, navigate to `www.professormesser.com/documents/python-sample-scripts.zip`.

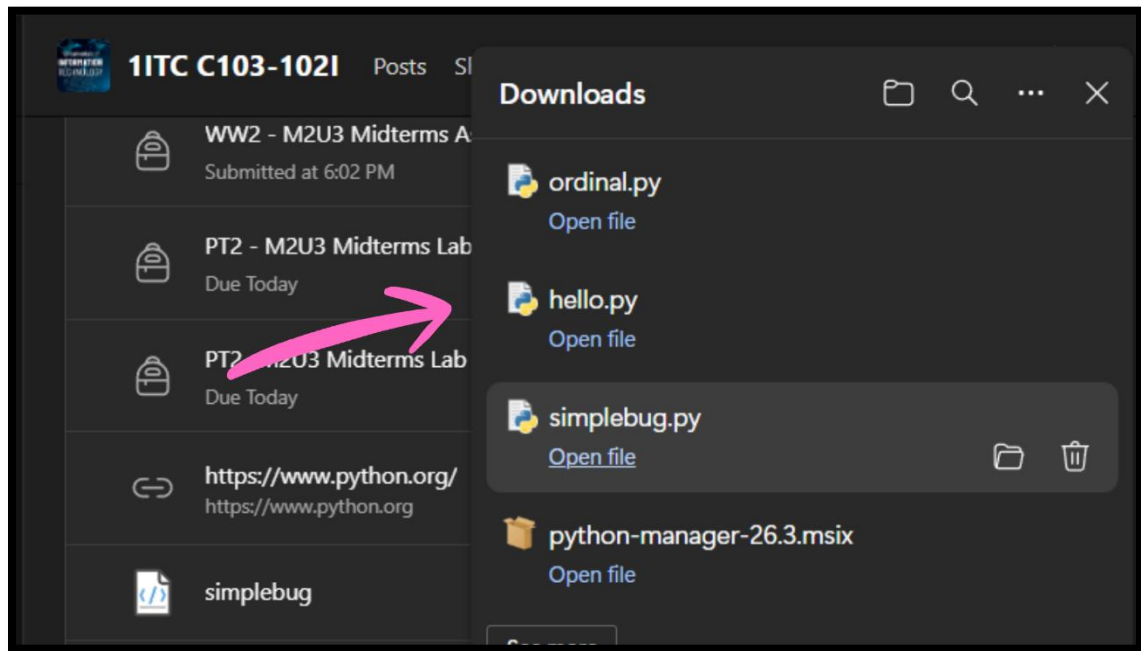


10. Click the **Save** button in the download bar at the bottom of the browser window.

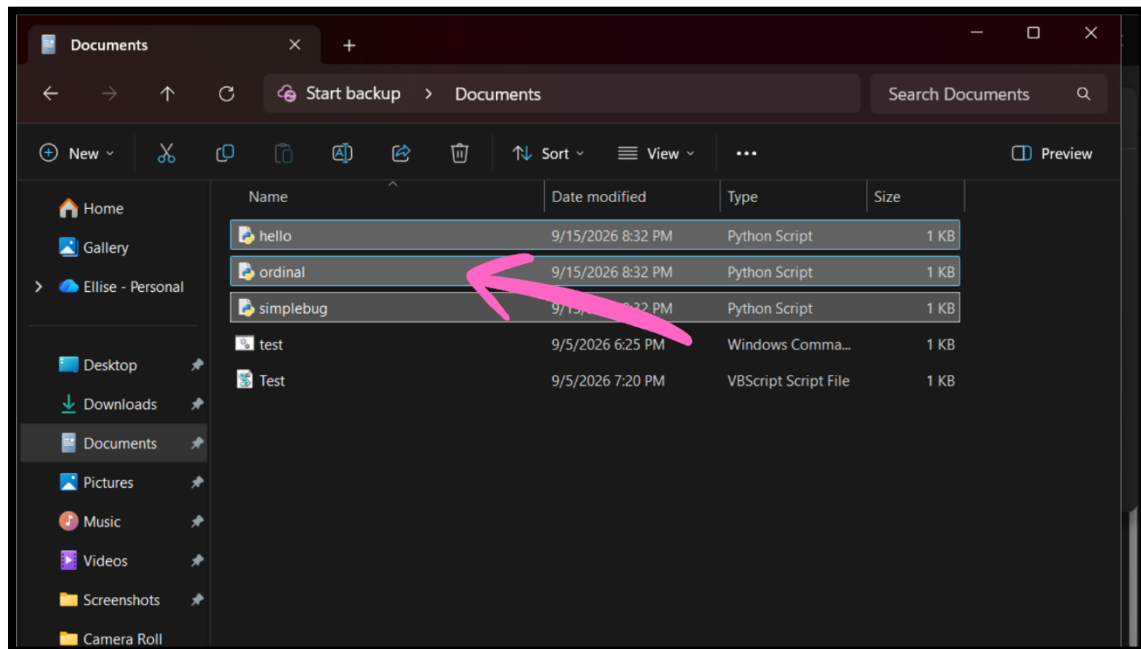




11. When the file has downloaded and the security scan is complete, in the download bar, click **Open folder**.

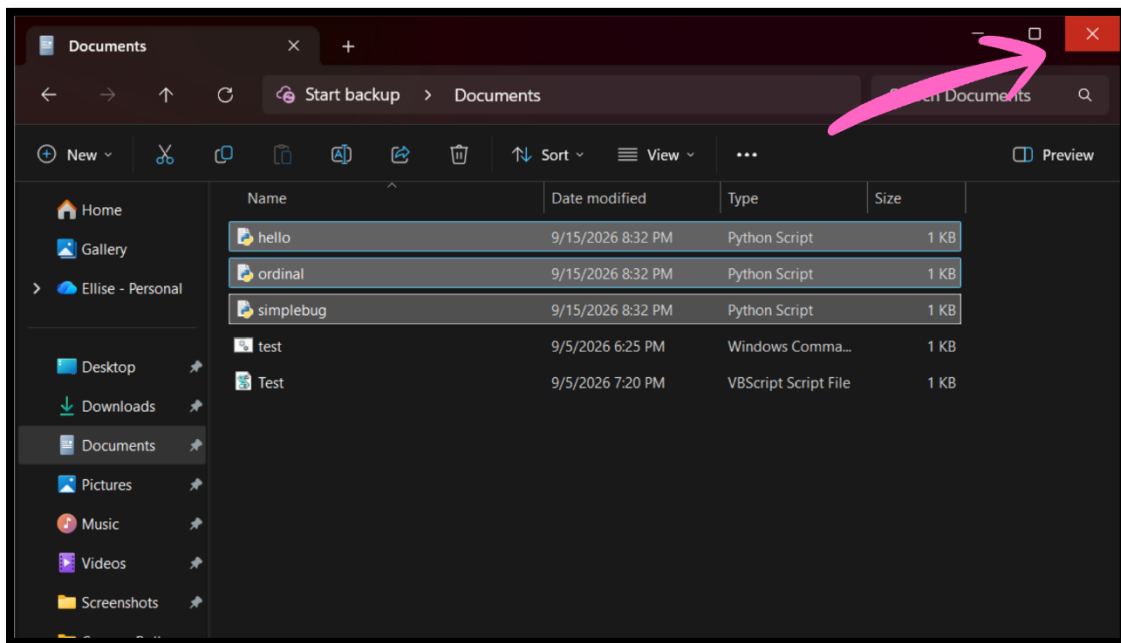


12. Drag the Python Scripts folder from the Downloads folder into your **Documents** folder.



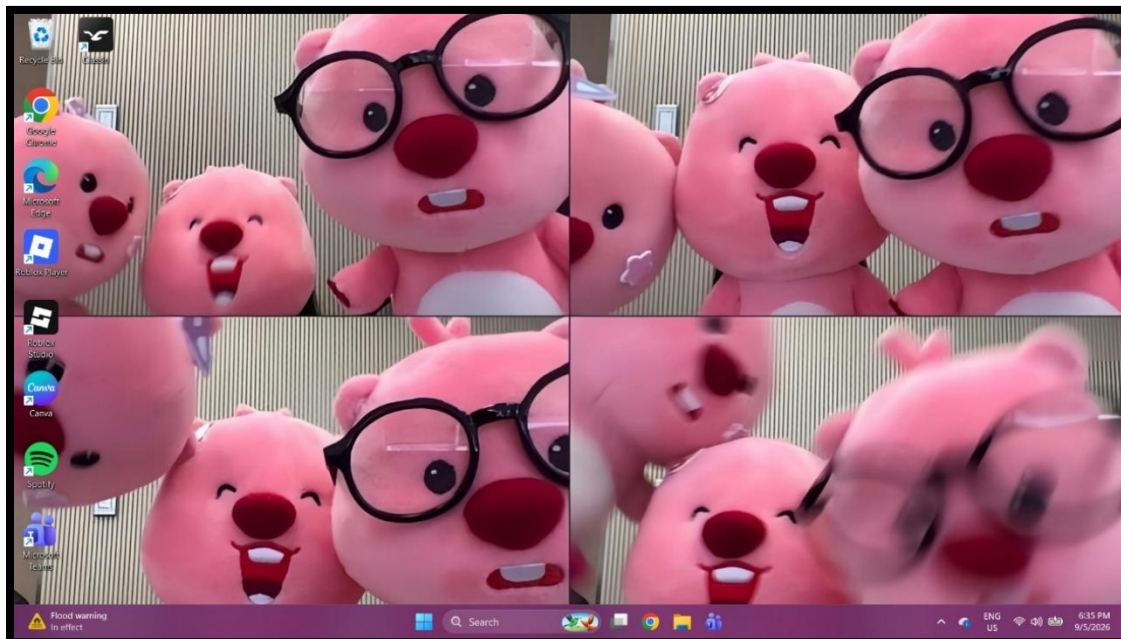


13. Close all open windows and apps.



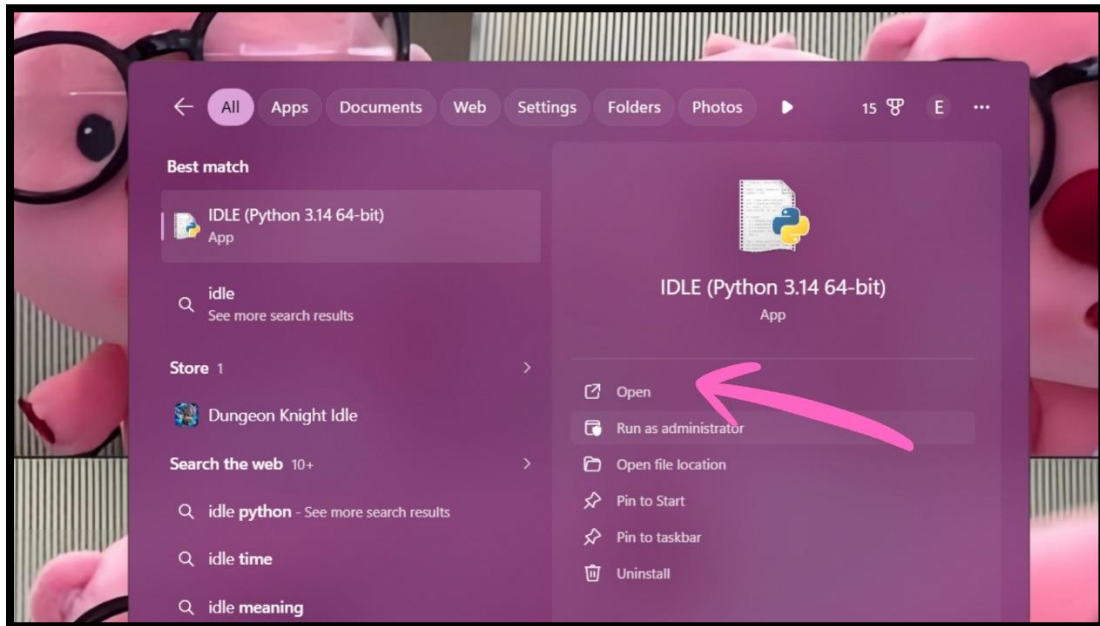
Part 2: Exploring a Simple Program

1. If necessary, start your computer and sign in.

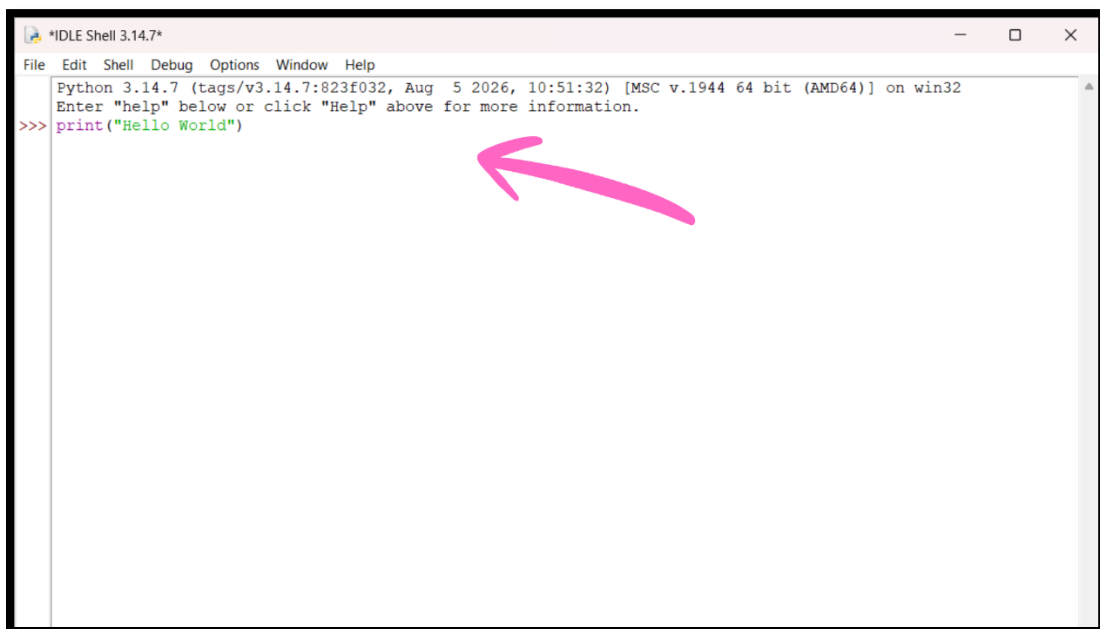




2. Click the **Start button** (on the taskbar), type `idle` in the search bar, and then click **IDLE (Python X.X)** from the search results (where X.X is your Python version, e.g., 3.13).



3. In the Python 3.X.X Shell window, type `print("Hello World")` and then press ENTER.





4. Notice that the shell responds by displaying the text in the parentheses.

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> print("Hello World")
Hello World
>>>
```

A pink arrow points to the output "Hello World" on the line following the print statement.

5. In the Python 3.X.X Shell window, type `name = input("Enter your name: ")` and then press ENTER.

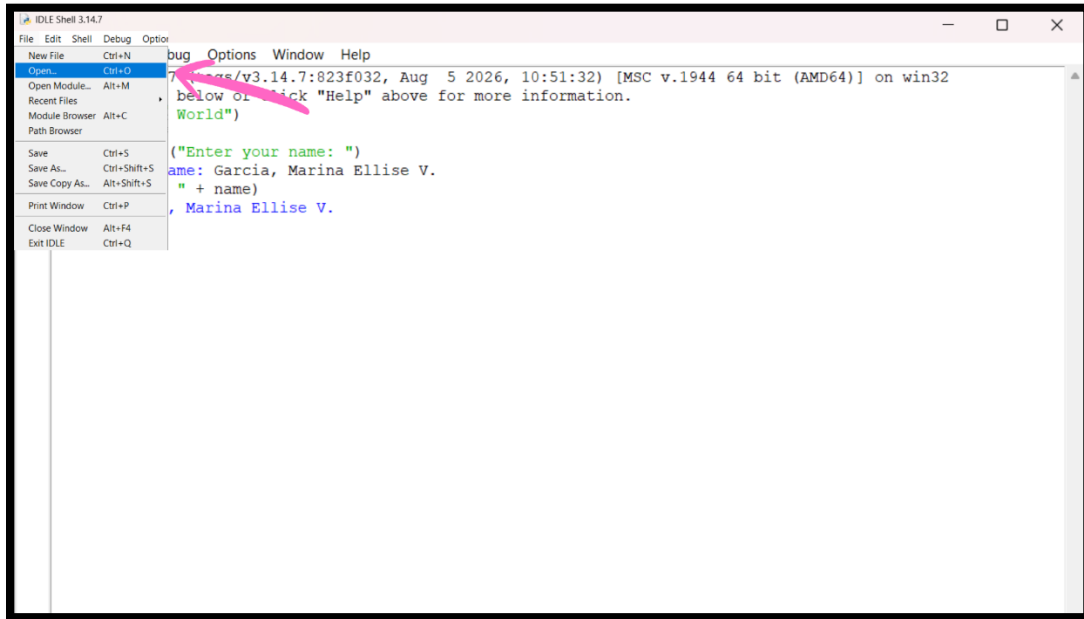
```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> print("Hello World")
Hello World
>>> name = input("Enter your name: ")
Enter your name:
```

A pink arrow points to the prompt "Enter your name:" on the line following the input statement.

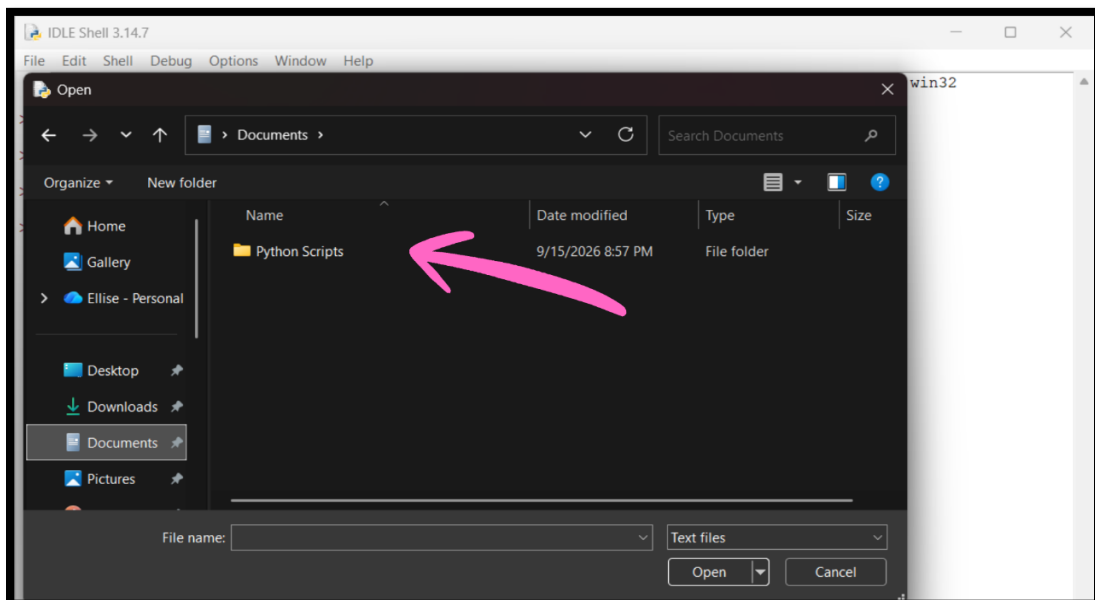
6. At the prompt, type your name and then press ENTER.
7. In the Python 3.X.X Shell window, type `print("Hello " + name)` and then press ENTER.



8. Notice that the shell responds by displaying the new text in the parentheses.
9. From the File menu, click **Open**.

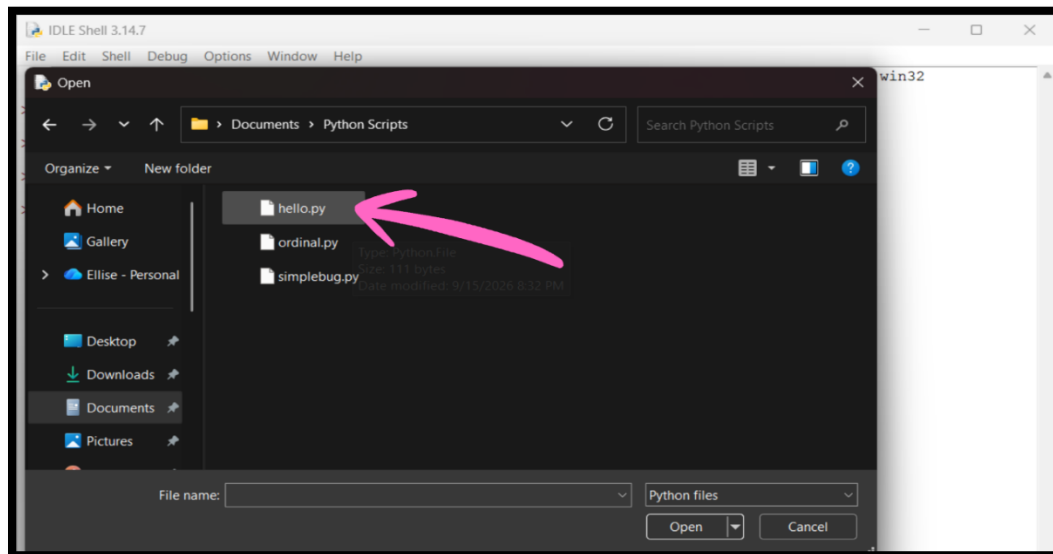


10. In the Open window, in the navigation pane, click **Documents**, and then double-click the **Python Scripts** folder.





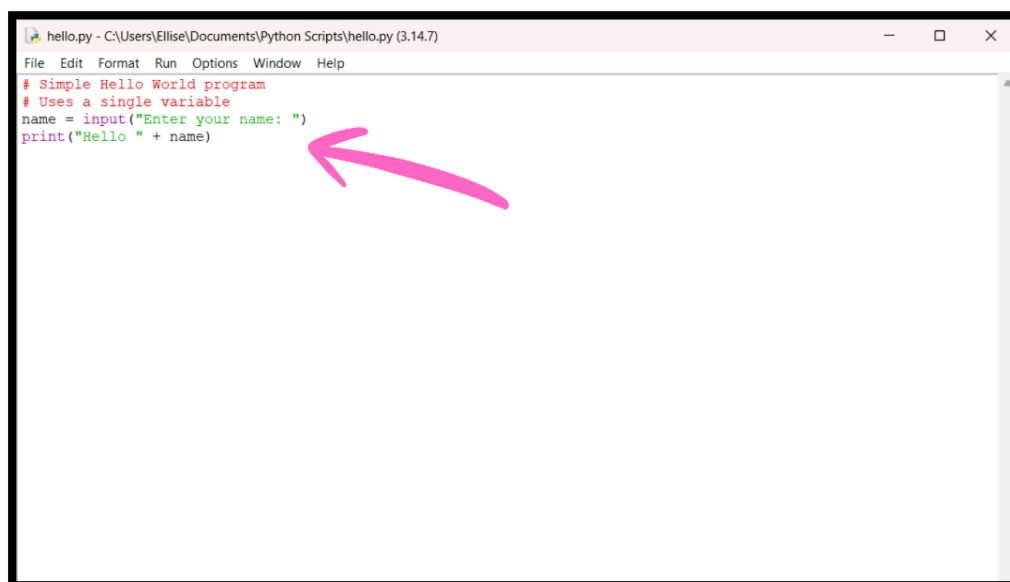
11. Double-click the `hello.py` file.



12. In the `hello.py` - `C:\Users\Documents\Python Scripts\hello.py` window, examine the code:

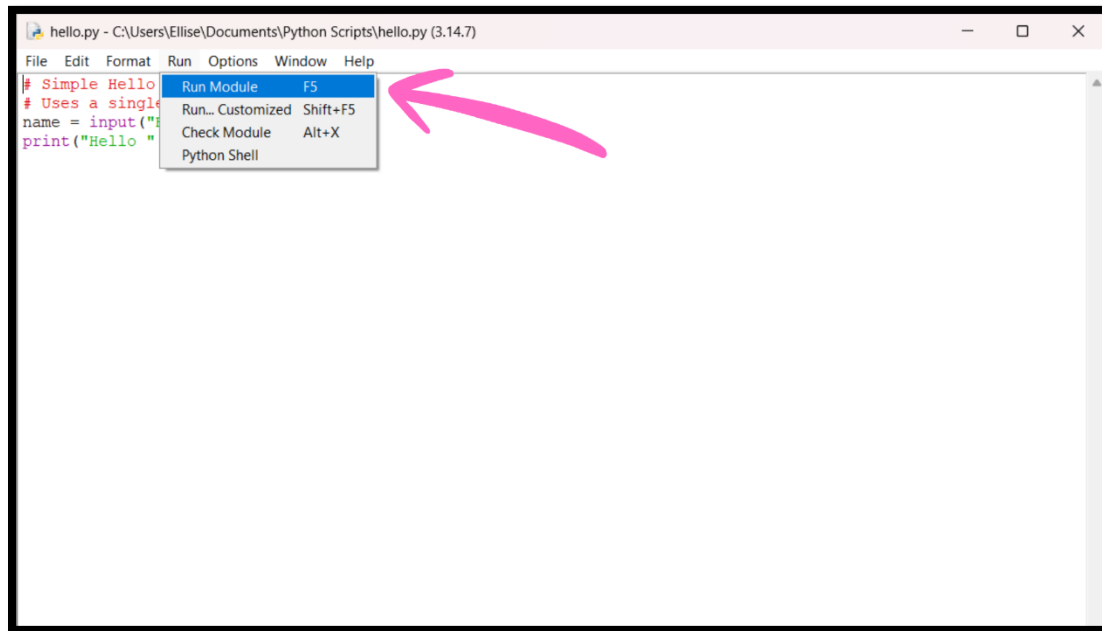
Python code:

```
# Simple Hello World program
# Uses a single variable
name = input("Enter your name: ")
print("Hello " + name)
```

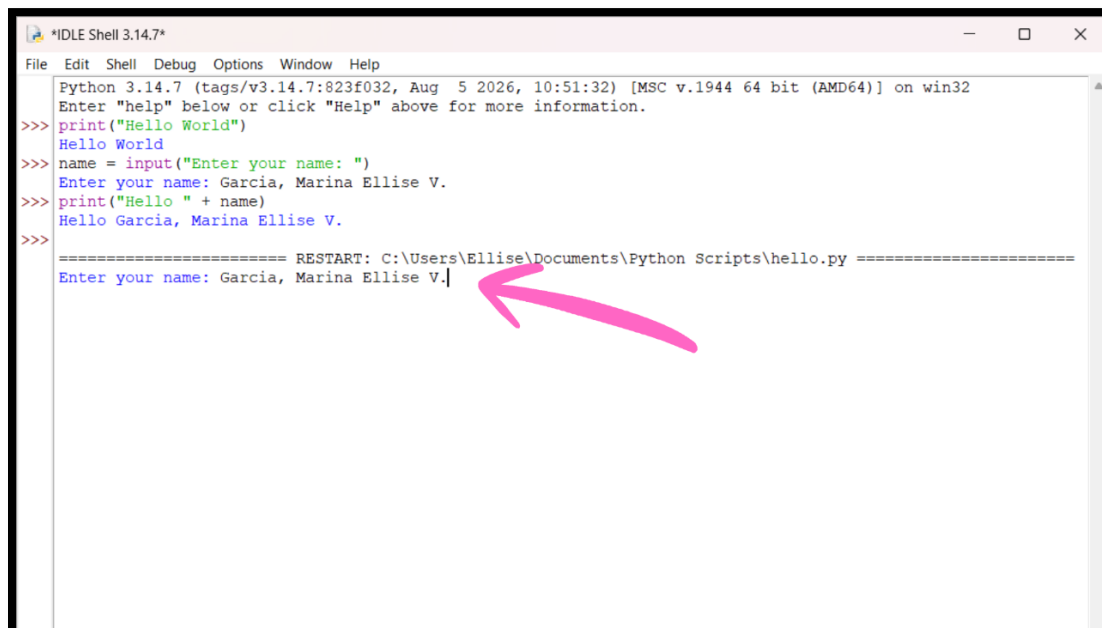




13. From the Run menu, click **Run Module**.



14. In the Python 3.X.X Shell window, type your name and then press ENTER.





15. Notice that the shell responds by displaying the new text in the parentheses.

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> print("Hello World")
Hello World
>>> name = input("Enter your name: ")
Enter your name: Garcia, Marina Ellise V.
>>> print("Hello " + name)
Hello Garcia, Marina Ellise V.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\hello.py =====
Enter your name: Garcia, Marina Ellise V.
Hello Garcia, Marina Ellise V.
>>> |
```

16. In the `hello.py` window, change the program as shown below:

Python code:

```
# Simple Hello World program
# Uses a single variable
name = input("Enter your name: ")
print("Hello " + name + " and welcome to the world of
Python")
for i in range(1, 10, 1):
    print(str(i) + "Squared is " + str(i*i))
```



- The new code adds a `for` loop to the program to calculate and display the square of the numbers 1 to 9. The loop starts at 1, increments by 1 at each iteration, and ends before it reaches 10. The value of `i` must be converted to a string before it is combined with other strings.

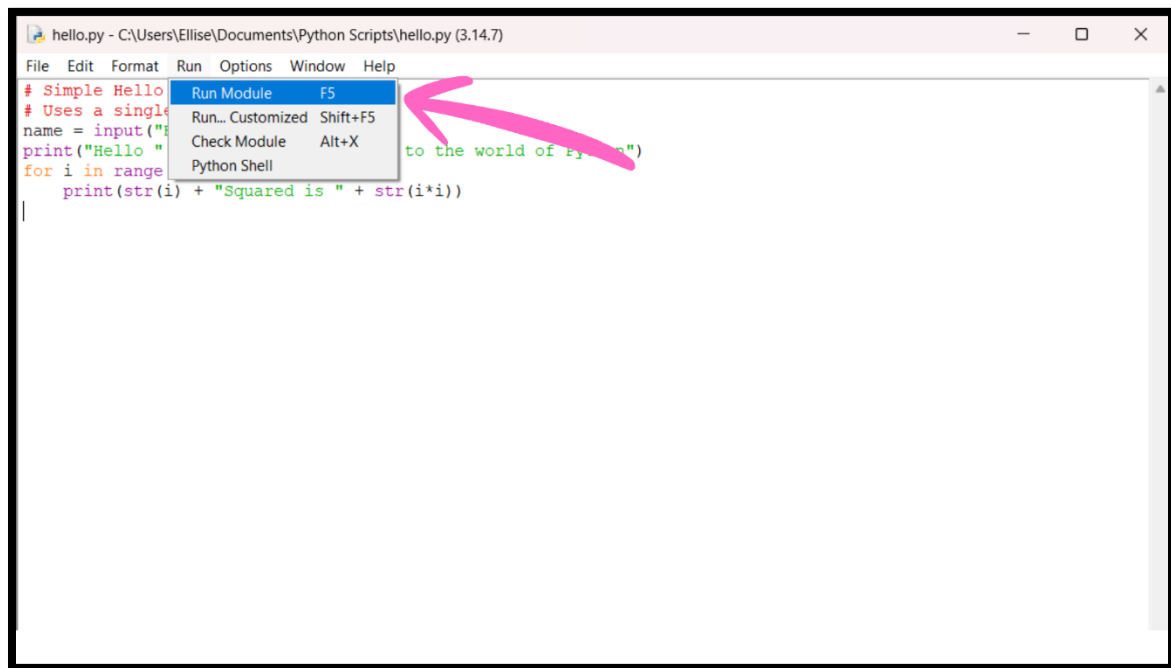
```
*hello.py - C:\Users\Ellise\Documents\Python Scripts\hello.py (3.14.7)*
File Edit Format Run Options Window Help
# Simple Hello World program
# Uses a single variable
name = input("Enter your name: ")
print("Hello " + name + " and welcome to the world of Python")
for i in range(1, 10, 1):
    print(str(i) + "Squared is " + str(i*i))
```

17. From the File menu, click Save.

```
*hello.py - C:\Users\Ellise\Documents\Python Scripts\hello.py (3.14.7)*
File Edit Format Run Options Window Help
New File      Ctrl+N
Open...       Ctrl+O
Open Module... Alt+M
Recent Files
Module Browser Alt+C
Path Browser
Save          Ctrl+S
Save As...    Ctrl+Shift+S
Save Copy As... Alt+Shift+S
Print Window  Ctrl+P
Close Window  Alt+F4
Exit IDLE     Ctrl+Q
```



18. From the Run menu, click ****Run Module**.



19. In the Python 3.X.X Shell window, type your name and then press ENTER.





20. Examine the output. Notice the loop continues to run until the condition is no longer valid.

```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> print("Hello World")
Hello World
>>> name = input("Enter your name: ")
Enter your name: Garcia, Marina Ellise V.
>>> print("Hello " + name)
Hello Garcia, Marina Ellise V.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\hello.py =====
Enter your name: Garcia, Marina Ellise V.
Hello Garcia, Marina Ellise V.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\hello.py =====
Enter your name: Gracia, Marina Ellise V.
Hello Gracia, Marina Ellise V. and welcome to the world of Python
1Squared is 1
2Squared is 4
3Squared is 9
4Squared is 16
5Squared is 25
6Squared is 36
7Squared is 49
8Squared is 64
9Squared is 81
>>>
```

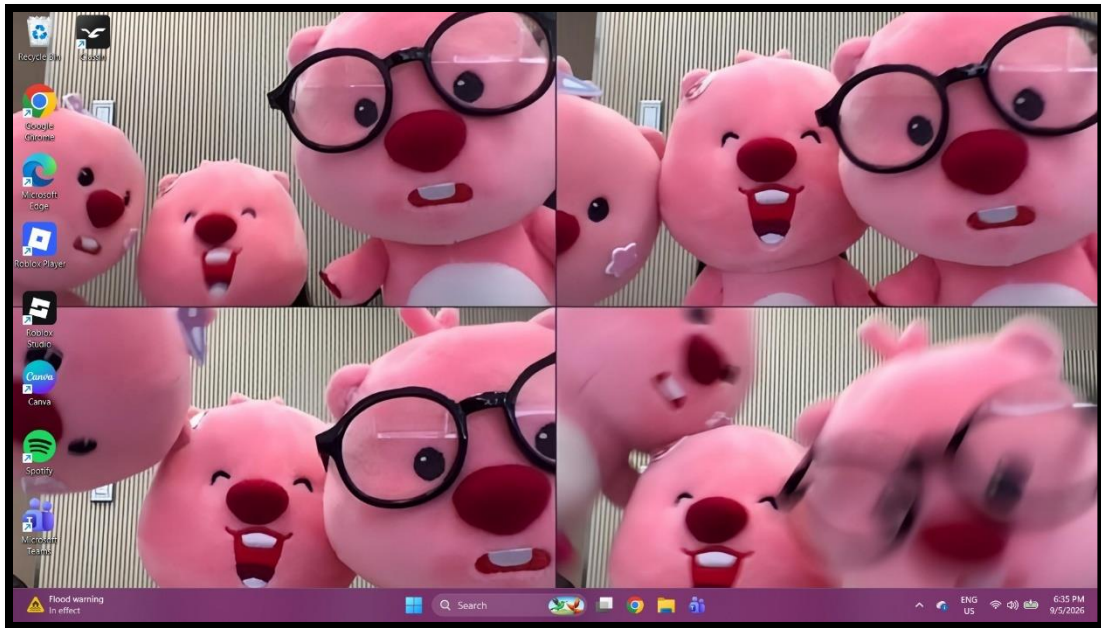
21. Close all open windows and apps.

```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" above for more information.
name: ")
arina Ellise V.
e V.
ESTART: C:\Users\Ellise\Documents\Python Scripts\hello.py =====
arina Ellise V.
e V.
ESTART: C:\Users\Ellise\Documents\Python Scripts\hello.py =====
arina Ellise V.
e V. and welcome to the world of Python
```

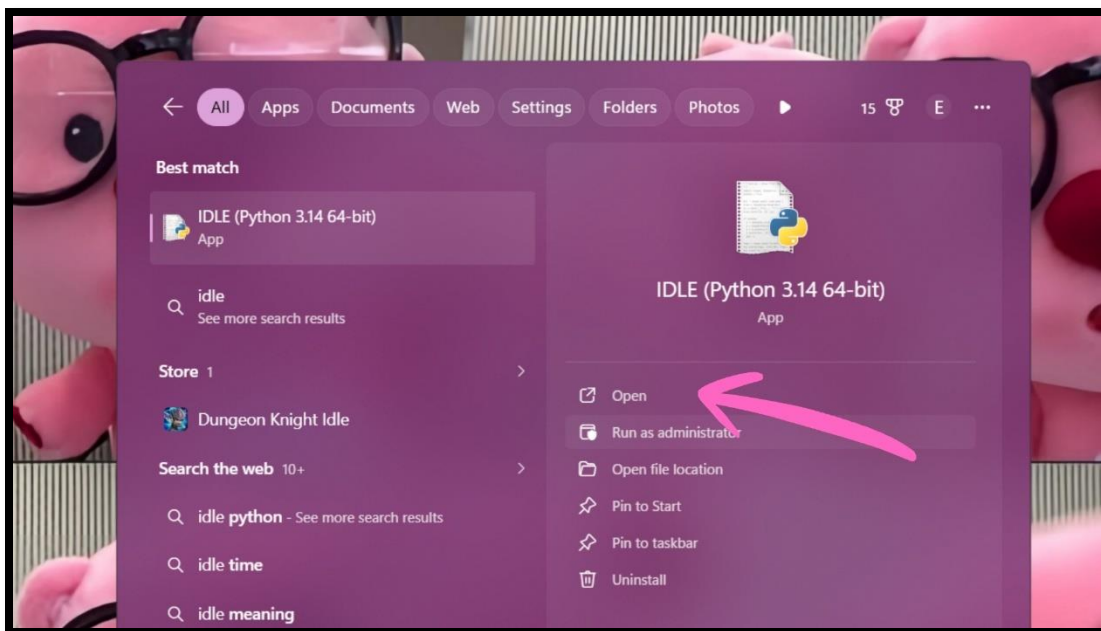


Part 3: Using the Debugger

22. If necessary, start your computer and sign in.

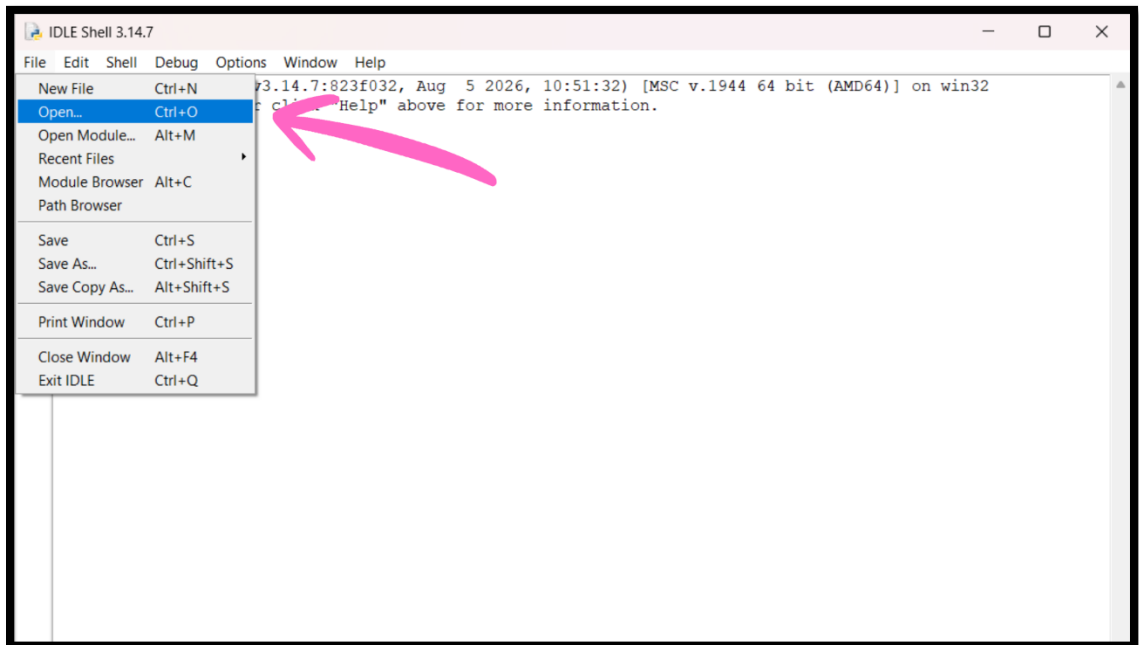


23. Click the Start button, type `idle` in the search bar, and then click **IDLE (Python X.X)** from the search results.

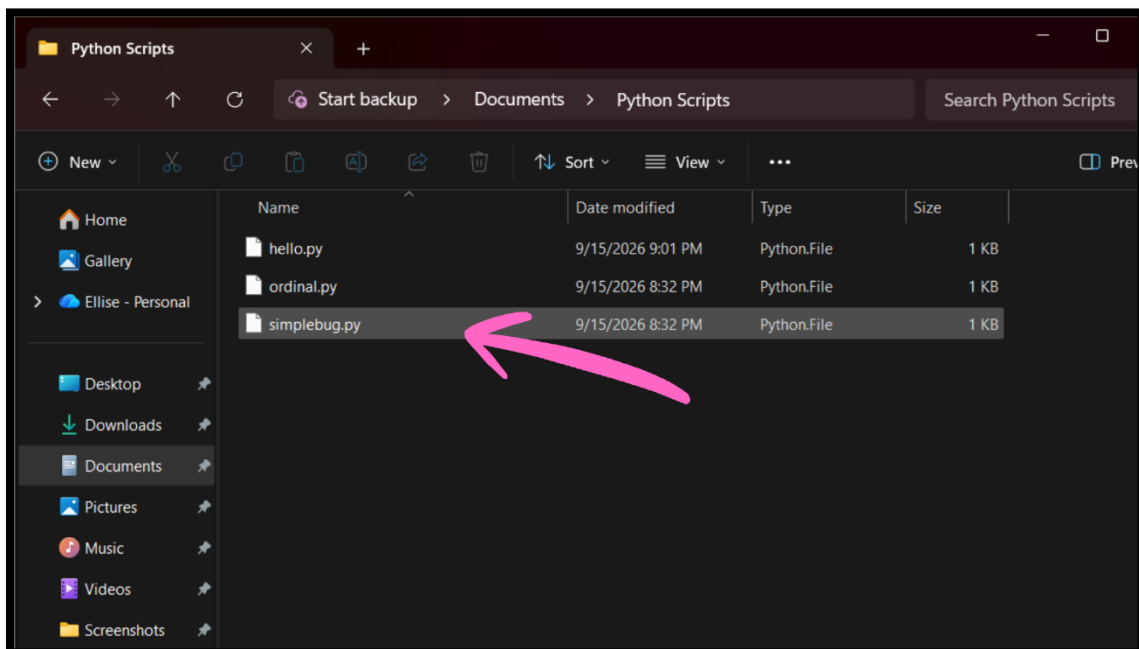




1. From the File menu, click **Open**.



24. In the Open window, in the navigation pane, click **Documents**, double-click the **Python Scripts** folder, and then double-click the `simplebug.py` file.





25. In the `simplebug.py` - `C:\Users\Documents\Python Scripts\simplebug.py` window, examine the code:

Python Code:

```
# Simple program with a logic error
# It attempts to add two numbers but makes a mistake
print("Start of program")
x = input("Enter first number: ")
y = input("Enter second number: ")
z = x + y
print("The total is " + str(z))
print("End of program")
```

The intention of the program is to add two numbers, but if you enter 2 and 3, the result is 23 rather than 5. This is because the input values are treated as strings rather than integers.

```
simplebug.py - C:\Users\Ellise\Documents\Python Scripts\simplebug.py (3.14.7)
File Edit Format Run Options Window Help
# Simple program with a logic error
# It attempts to add two numbers but makes a mistake
print("Start of program")
x = input("Enter first number: ")
y = input("Enter second number: ")
z = x + y
print("The total is " + str(z))
print("End of program")
```

A screenshot of the IDLE Shell 3.14.7 window. The window has a menu bar with File, Edit, Shell, Options, Window, and Help. The main text area shows the output of a Python script. The script starts with a prompt >>>, followed by a restart message: ===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====. The script then prints "Start of program", prompts for "Enter first number: 1", prompts for "Enter second number: 24", prints "The total is 124", and ends with "End of program". The prompt >>> is shown again at the bottom.

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
>>>
```

From the Debug menu, click Debugger.

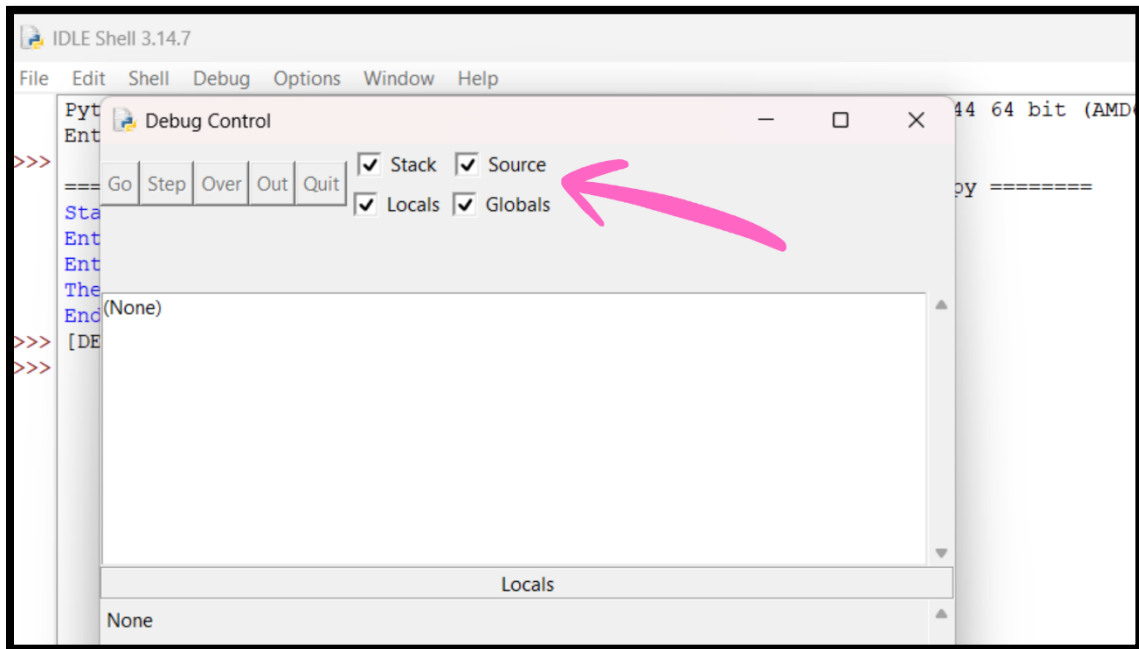
A screenshot of the IDLE Shell 3.14.7 window, similar to the previous one, but with the Debug menu open. The menu options are: Go to File/Line, Debugger, Stack Viewer, and Auto-open Stack Viewer. A pink arrow points to the "Debugger" option. The main text area shows the same output as the previous screenshot.

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

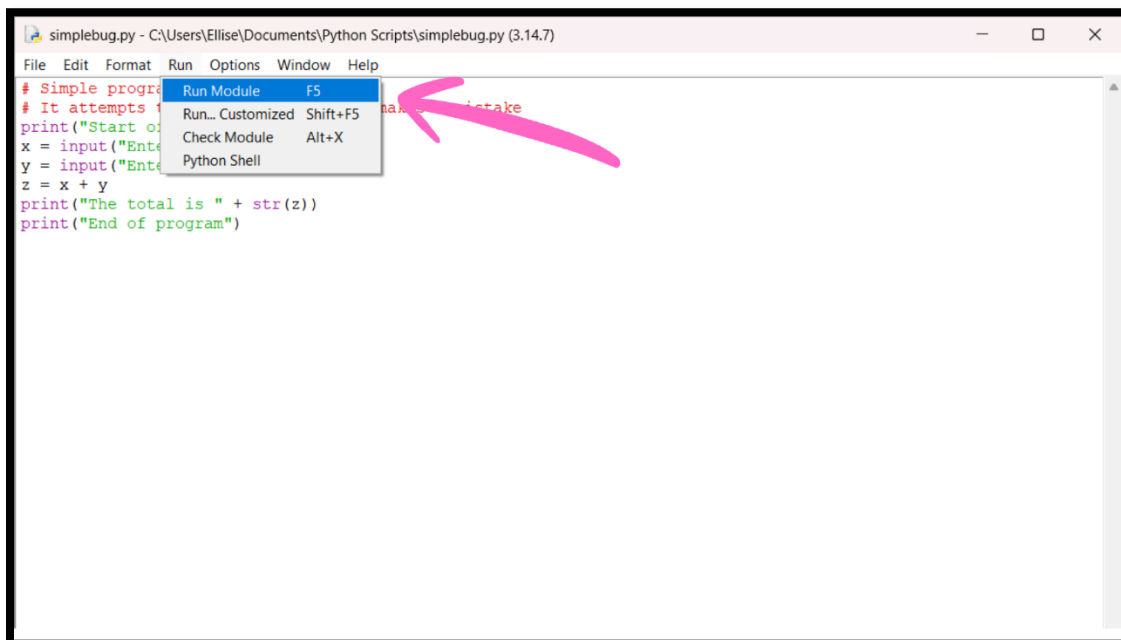
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
>>>
```



2. In the Debug Control window, ensure that **Source** and **Globals** are checked.



3. From the Run menu, click **Run Module**.





4. In the Python 3.X.X Shell window, at the prompt for the first number, enter 2.

```
*IDLE Shell 3.14.7*
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
>>> [DEBUG ON]
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
2
```

5. In the Debug Control window, click **Step**.
- The green arrow in the `simplebug.py` window advances to the next line of code, and the Globals column in the Debug Control window displays the new value of the variable `x`.

```
*IDLE Shell 3.14.7*
File Edit Shell Debug Options Window Help
Pyt
Ent
>>>
=== Go Step | Out Quit | [x] Stack [x] Source
Sta [x] Locals [x] Globals
Ent simplebug.py:3: <module>()
Ent
Ent
The
End
Ent
>>> [DE
>>>
=====
2
```

Debug Control

Locals

None

Globals

__builtins__ <module 'builtins' (built-in)>



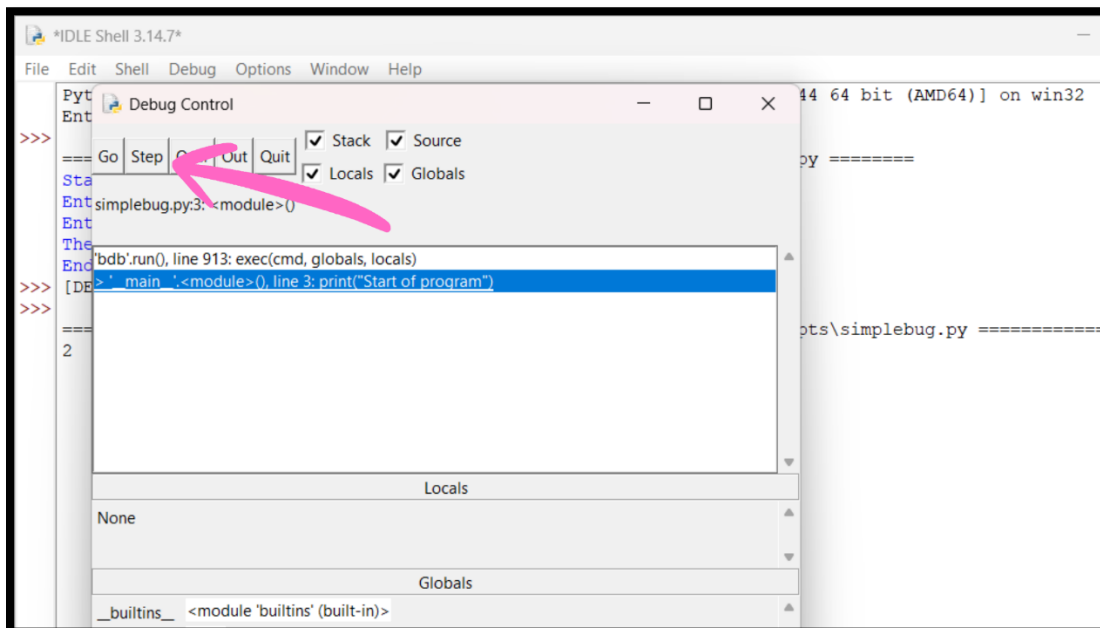
6. In the Python 3.X.X Shell window, at the prompt for the second number, enter 3.

```
*IDLE Shell 3.14.7*
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
[DEBUG ON]

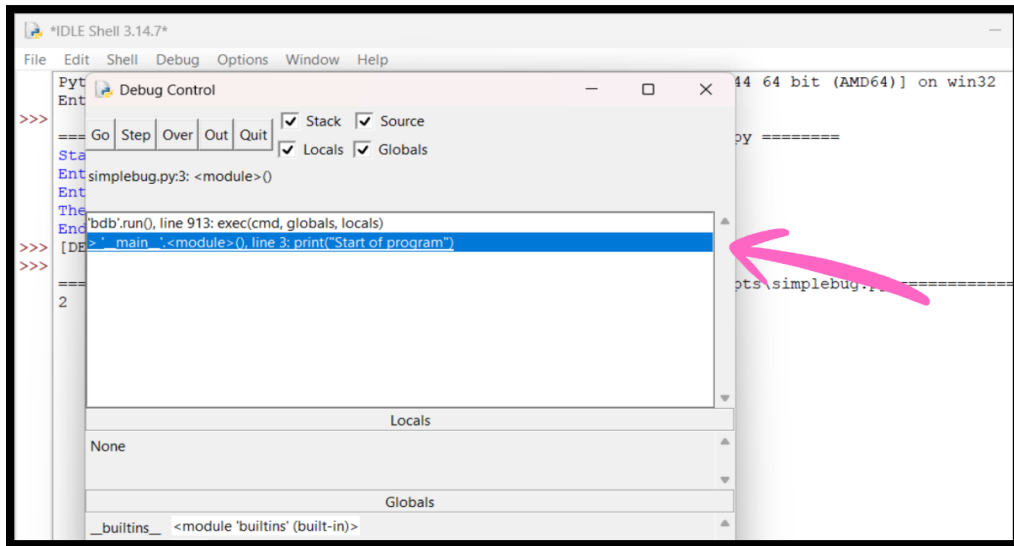
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 2
3
```

7. In the Debug Control window, click Step.





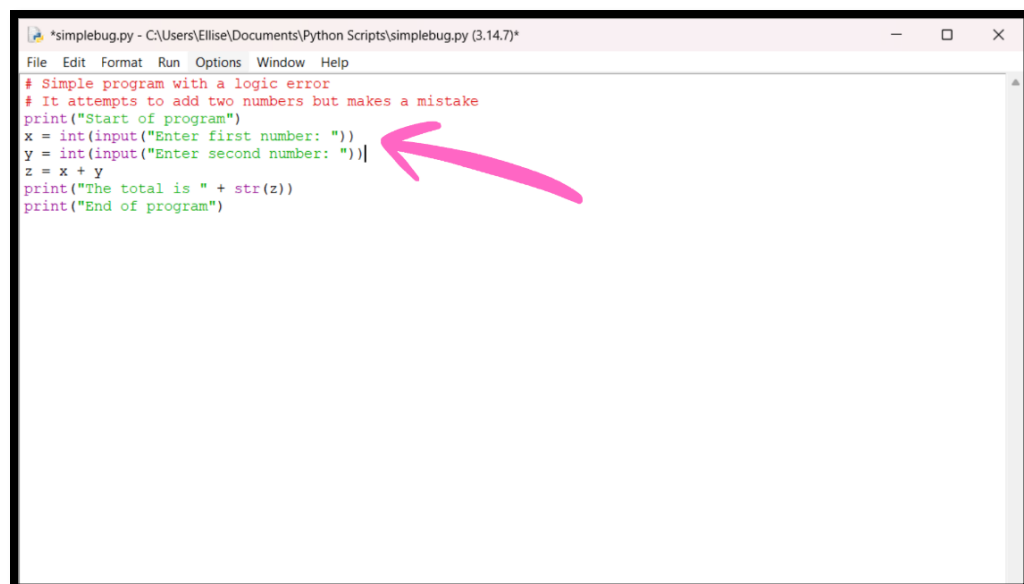
8. The green arrow in the `simplebug.py` window advances to the next line of code, and the Globals column in the Debug Control window displays the new value of the variable `z` ('23'). This confirms that `x` and `y` are being treated as strings and concatenated rather than added together.



9. In the `simplebug.py` window, change lines 3 and 4 as follows:

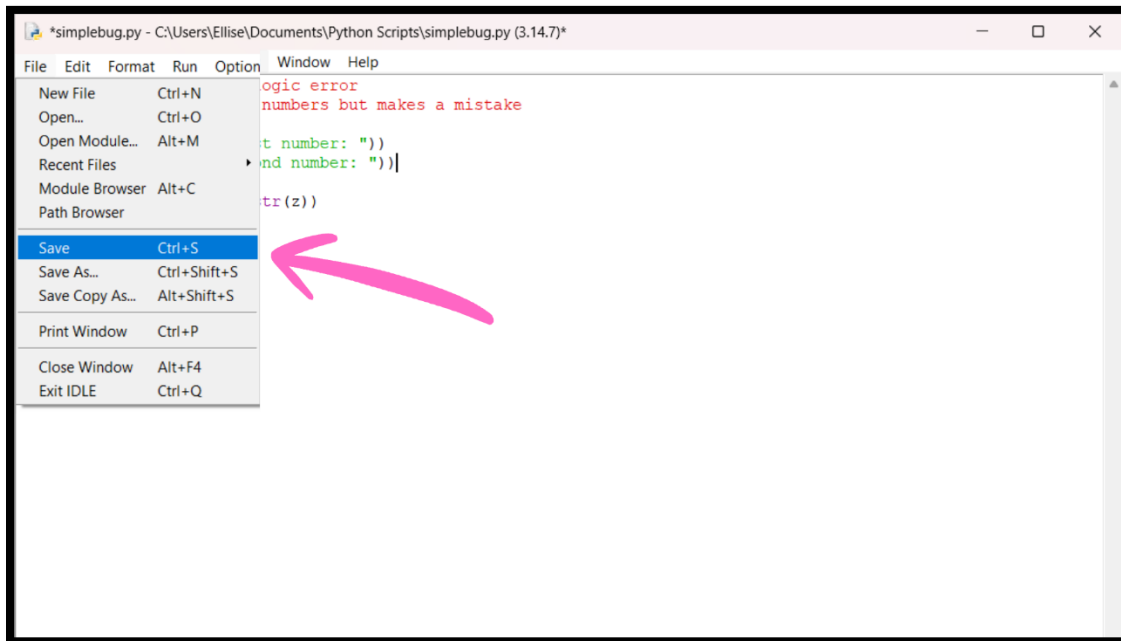
Python Code:

```
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
    ○ This changes the input values from strings to integers using Python's
      int() function.
```

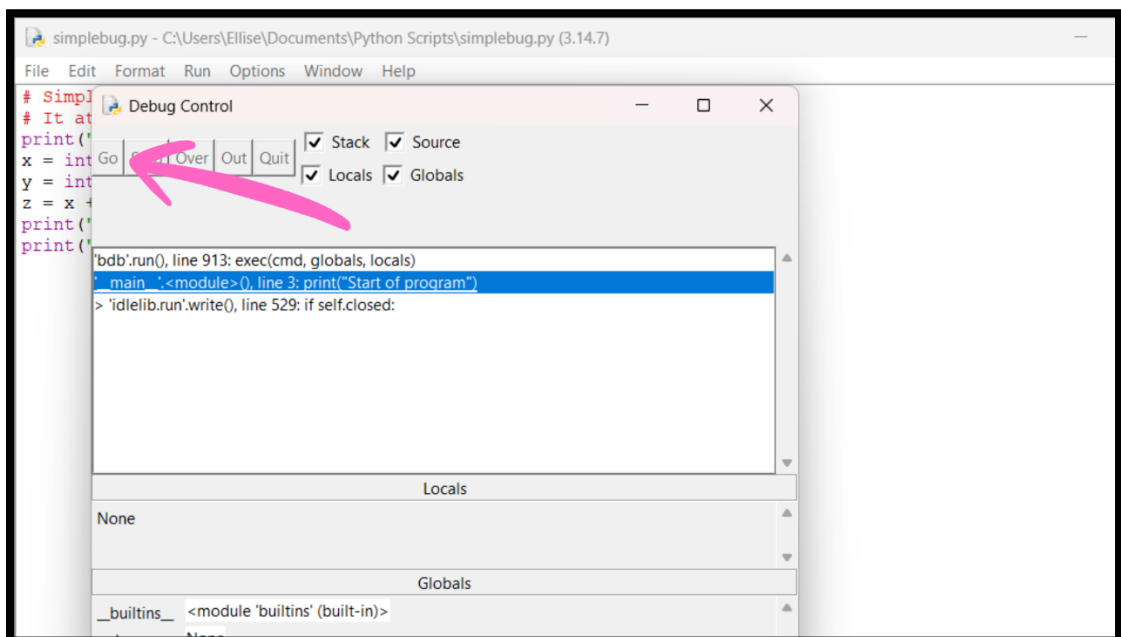




10. From the File menu, click Save.



11. From the Debug menu, click Go.





12. In the Python 3.X.X Shell window, at the prompt for the first number, enter 2.

```
*IDLE Shell 3.14.7*
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
[DEBUG ON]
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 2
Enter second number: |
```

13. At the prompt for the second number, enter 3.

```
*IDLE Shell 3.14.7*
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 1
Enter second number: 24
The total is 124
End of program
[DEBUG ON]
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py =====
Start of program
Enter first number: 2
Enter second number: 3
```



14. Notice that the total is now 5.

```
>>> Enter first number: 1
>>> Enter second number: 24
>>> The total is 124
>>> End of program
>>> [DEBUG ON]
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py
>>> Start of program
>>> Enter first number: 2
>>> Enter second number: 3
>>> The total is 23
>>> End of program
>>> [DEBUG ON]
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\simplebug.py
>>> Start of program
>>> Enter first number: 2
>>> Enter second number: 3
>>> The total is 5
>>> End of program
>>> [DEBUG ON]
>>>
```

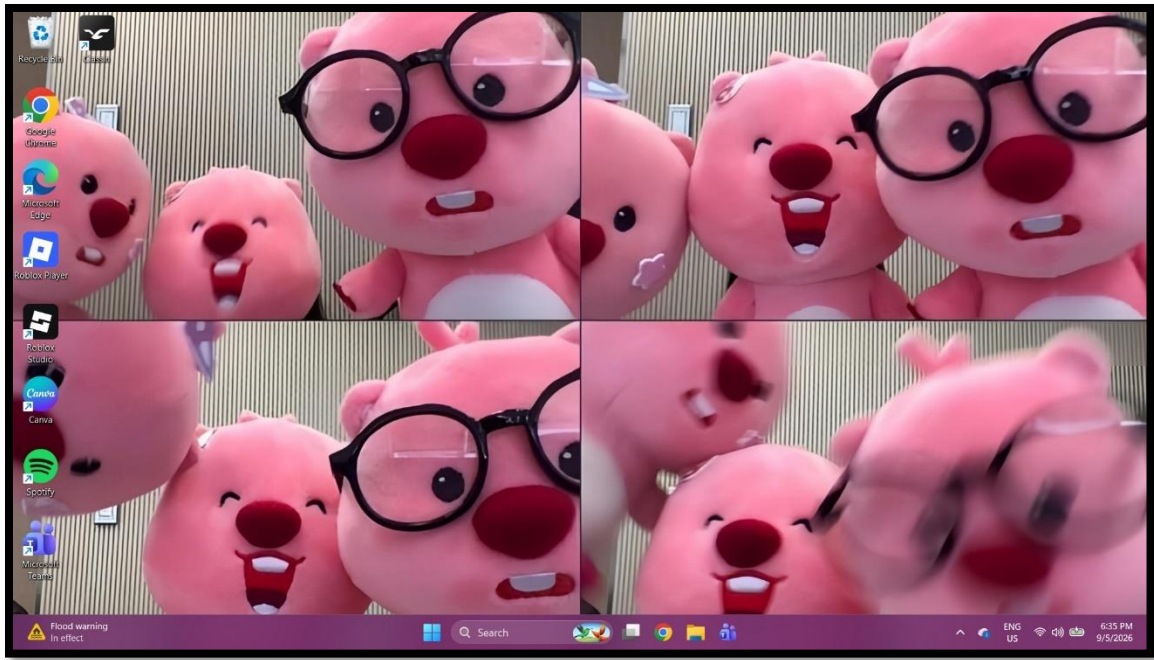
15. Close all open windows and apps.



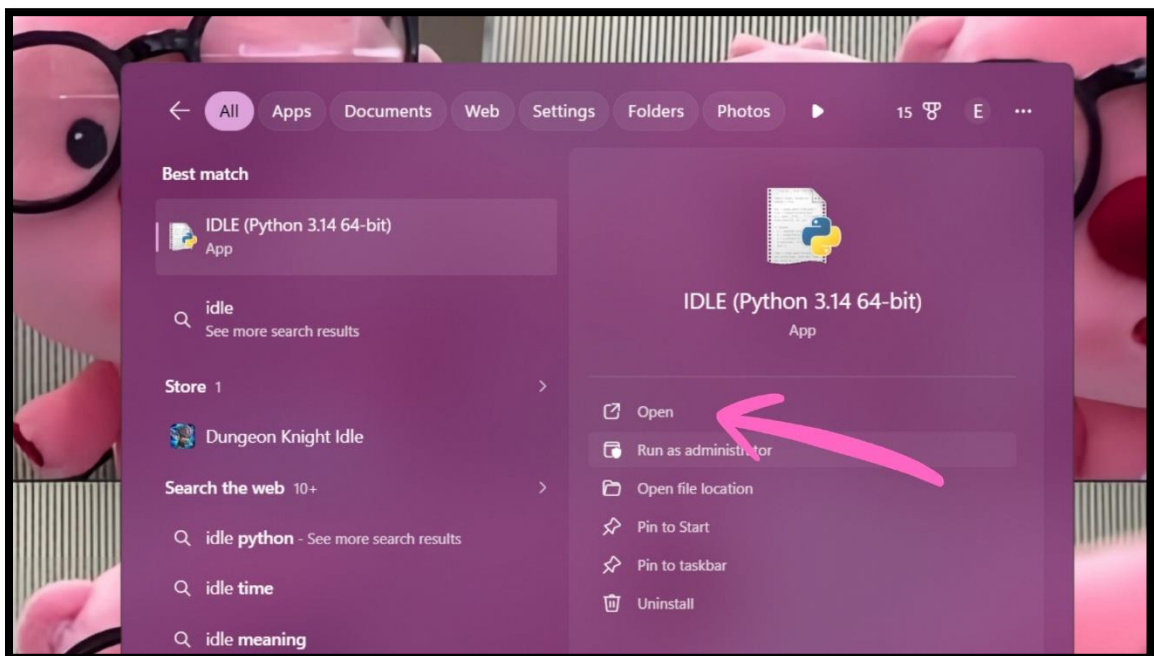


Part 4: Using a Function

1. If necessary, start your computer and sign in.

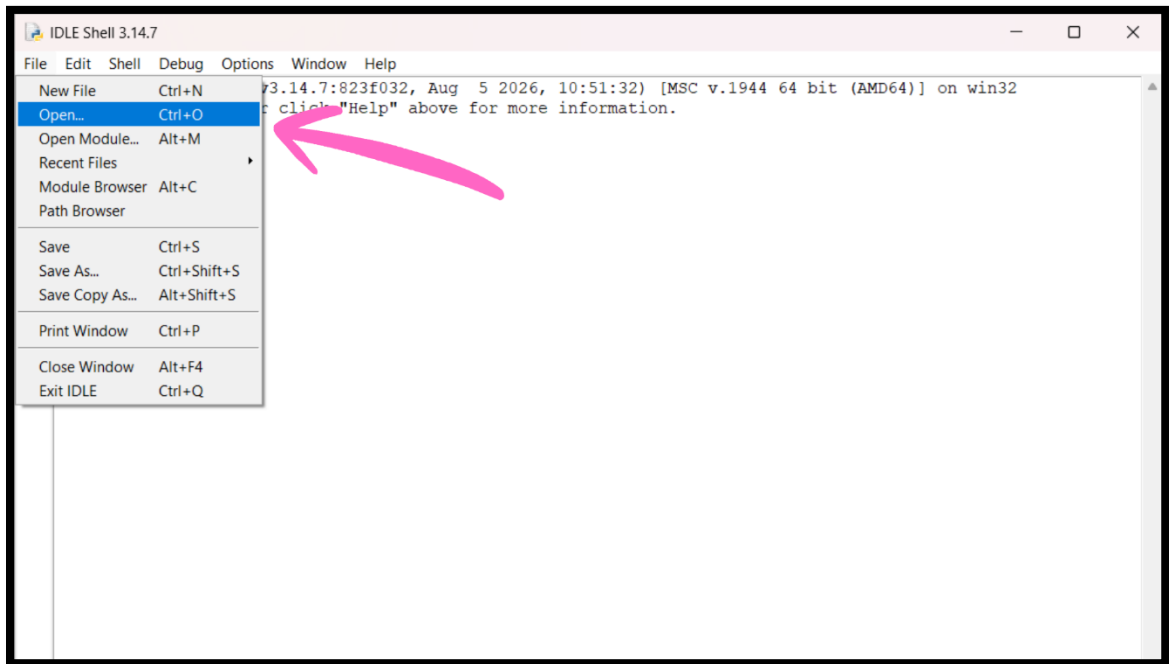


2. Click the **Start** button, type idle in the search bar, and then click **IDLE (Python X.X)** from the search results.

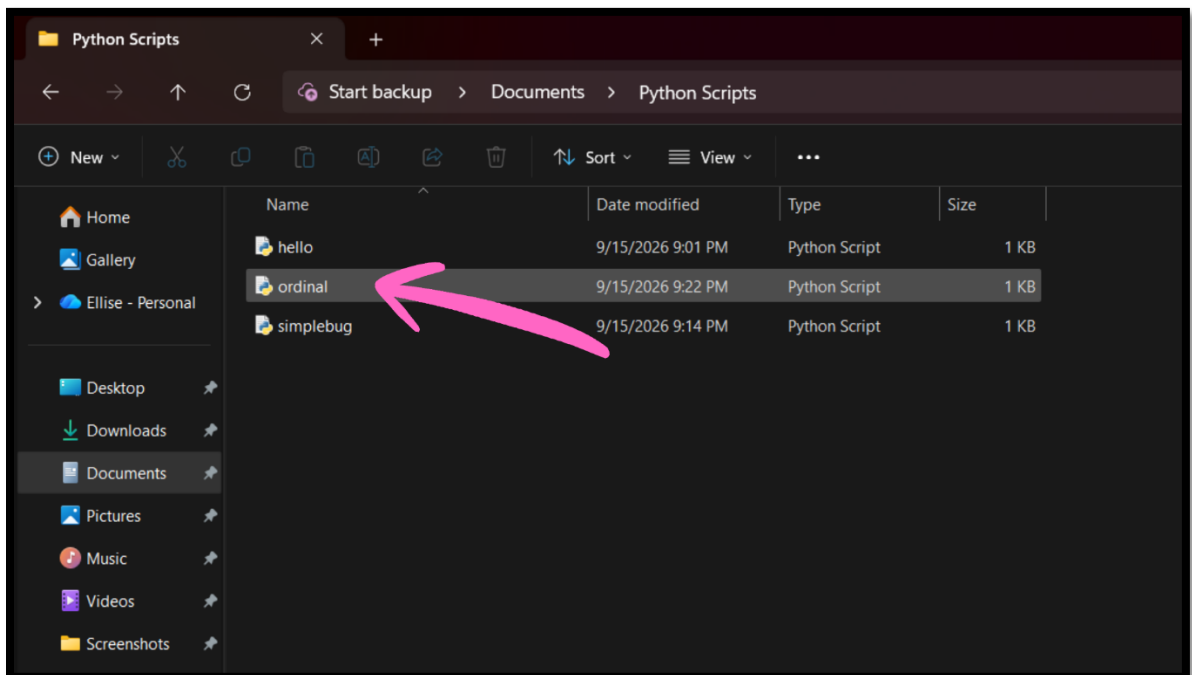




3. From the File menu, click **Open**.



4. In the Open window, in the navigation pane, click **Documents**, double-click the **Python Scripts** folder, and then double-click the `ordinal.py` file.





5. In the `ordinal.py` - `C:\Users\Documents\Python Scripts\ordinal.py` window, examine the code:

Python Code:

```
# Ordinal suffix program with a function and dictionary
SUFFIX_DICT = {
    0: 'th',
    1: 'st',
    2: 'nd',
    3: 'rd',
    4: 'th',
    5: 'th',
    6: 'th',
    7: 'th',
    8: 'th',
    9: 'th'
}

def ordinal_suffix(i):
    if i >= 11 and i <= 13:
        return 'th'
    if i > 99:
        i = int(str(i)[-2:]) # Truncate to last two digits

    return SUFFIX_DICT.get(i % 10, 'th')

def main():
    print("Start of program")
    for i in range(1, 3 + 1, 1):
        s = str(i) + ordinal_suffix(i)
        print(s)
    print("End of program")

if __name__ == '__main__':
    main()
```



- The program defines a function called `ordinal_suffix()` that returns the correct suffix for a number (e.g., 1st, 2nd, 3rd, 4th).

```
ordinal.py - C:\Users\Ellise\Documents\Python Scripts\ordinal.py (3.14.7)
File Edit Format Run Options Window Help
# Ordinal suffix program with a function and dictionary
SUFFIX_DICT = {
    0: 'th',
    1: 'st',
    2: 'nd',
    3: 'rd',
    4: 'th',
    5: 'th',
    6: 'th',
    7: 'th',
    8: 'th',
    9: 'th'
}

def ordinal_suffix(i):
    if i >= 11 and i <= 13:
        return 'th'
    if i > 99:
        i = int(str(i)[-2:]) # Truncate to last two digits
    return SUFFIX_DICT.get(i % 10, 'th')

def main():
    print("Start of program")
    for i in range(1, 3 + 1, 1):
        s = str(i) + ordinal_suffix(i)
        print(s)
    print("End of program")

if __name__ == '__main__':
    main()
```

- It uses a dictionary (`SUFFIX_DICT`) to store the common suffixes for numbers ending in 0-9.

```
def ordinal_suffix(i):
    if i >= 11 and i <= 13:
        return 'th'
    if i > 99:
        i = int(str(i)[-2:]) # Truncate to last two digits
    return SUFFIX_DICT.get(i % 10, 'th')
```



- The `if __name__ == '__main__':` block ensures that the `main()` function is called only when the script is executed directly (not when it's imported as a module).



```
if __name__ == '__main__':  
    main()
```

6. Examine the `ordinal_suffix()` function. Ensure you understand what it does:

- In the first if statement, we check whether the integer is over two digits long. If it is, we truncate it, using functions to convert between integer and string and back. For example, if the value of `i` is 112, this statement will convert it to 12. The code snippet `[-2:]` is a slice, which is the method used in Python to return part of a string (in this case the last two characters). We're doing this because we only need to evaluate the values from 0 to 99 to determine the appropriate suffix.



```
def ordinal_suffix(i):  
    if i >= 11 and i <= 13:  
        return 'th'
```



- The second `if` statement checks whether `i` has the value of 11, 12, or 13 because in these special cases, the suffix is "th" not "11st" or "12nd." The code uses the comparison operator less than (`<`) to accomplish this.

```
if i > 99:  
    i = int(str(i)[-2:]) # Truncate to last two digits
```

- The `else` statement returns the correct suffix in any other case. It does this by using the mod operator (`%`). Mod returns the remainder after division. For example, if `i` is 2 then the remainder after division by 10 is also 2. If `i` is 22, the remainder is still 2. The `get()` function is used to lookup this remainder value in the `SUFFIX_DICT` list. If the value is not found (if it is zero or four for instance), "th" is returned by default.

```
return SUFFIX_DICT.get(i % 10, 'th')
```



7. Use F5 to run the new program and test that it returns correctly formatted suffixes in the output.

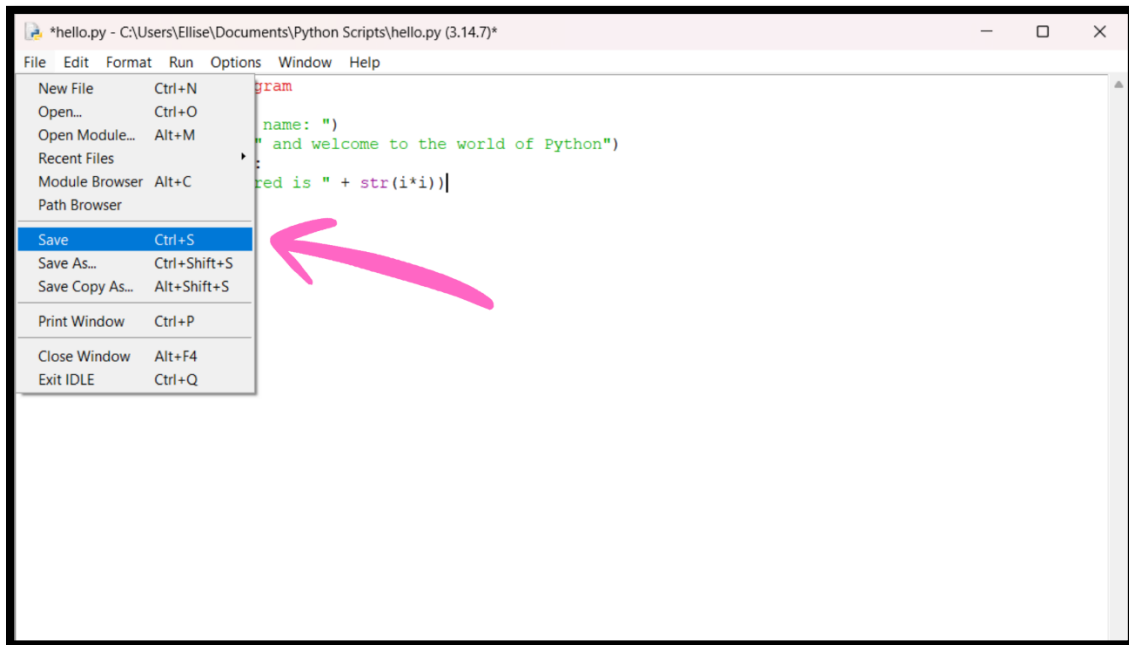
```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Ellise\Documents\Python Scripts\ordinal.py =====
Start of program
1st
2nd
3rd
End of program
>>>
```

8. Change the value of `range(1, 3+1, 1)` to test different integers. For example, try `range(109, 124, 1)`.

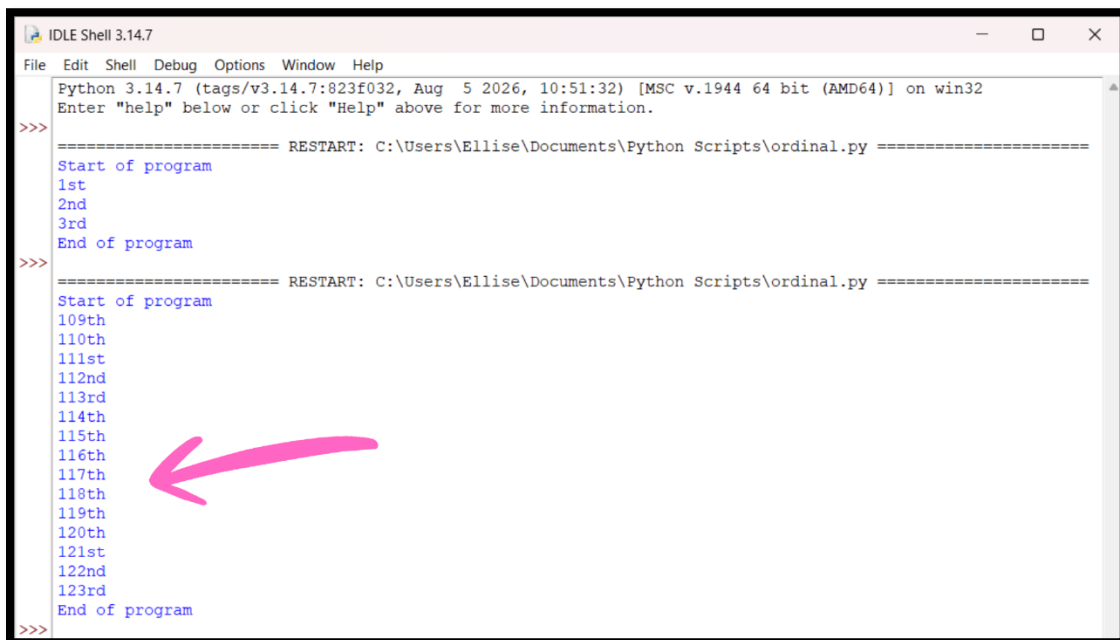
```
def main():
    print("Start of program")
    for i in range(109, 124, 1):
        s = str(i) + ordinal_suffix(i)
        print(s)
    print("End of program")
```



9. From the File menu, click **Save**.

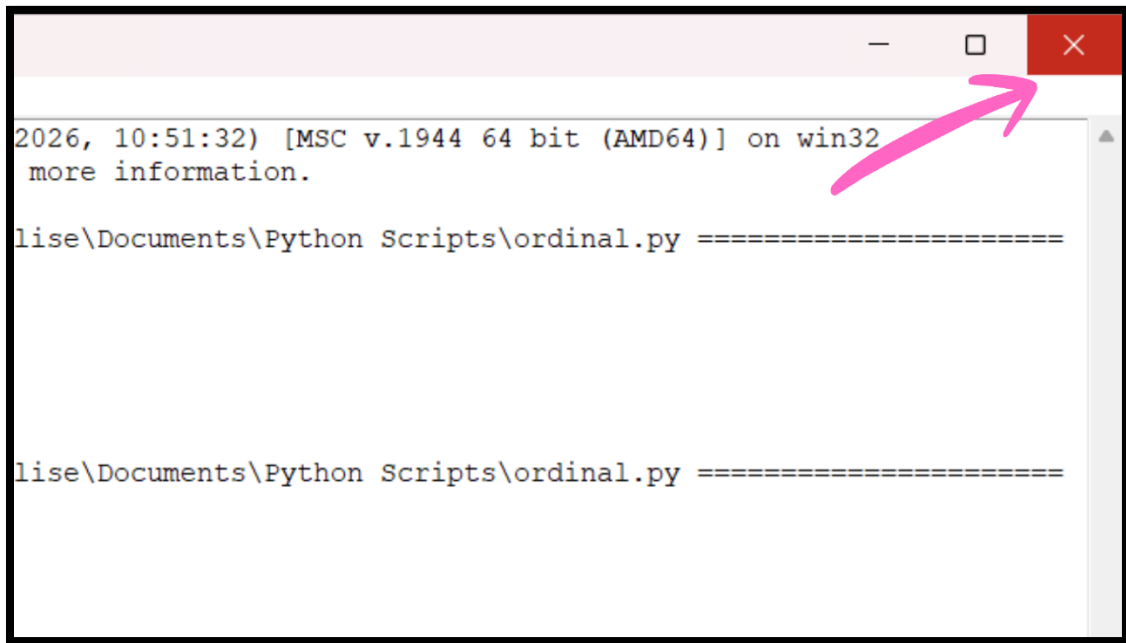


10. Press **F5** to run the program and examine the output.





11. Close all open windows and apps.





V. REFLECTION:

Upon successful completion of "Lab 2.3.9: Using Programming Tools," write a comprehensive reflection (between 150-250 words) addressing the following points:

- **Learning Outcomes:** What new knowledge or skills did you acquire or reinforce regarding the use of a scripting programming language like Python?
- **Challenges and Solutions:** Describe any difficulties or unexpected issues you encountered during the lab. How did you overcome them, or what steps would you take to resolve them in the future?
- **Real-world Relevance:** How do the skills and understanding gained in this lab relate to real-world scenarios, either in your personal life or a professional IT context?
- **Critical Thinking:** If you had to use an internet image for any step, explain why you couldn't perform the action physically and how finding the image contributed to your understanding.

This activity helped me improve my understanding of the Python programming language. I learned to work with variables, loops, functions, dictionaries, and debugging. The task that required me to use the print function to get the sum of two numbers was challenging. The problem was that Python printed two numbers as a string. The issue was solved by using integer to convert numbers. Thus, I have learned the importance of using different data types when programming in Python. Furthermore, I have learned that debugging is a helpful tool that allows to find and fix errors in the code. As a result, I am more confident in my programming skills and have got a better understanding of how to use Python in IT field.